PREFACE

The structures that shape us.

In the 1920s, San Francisco sat like a fortress at the end of a continent, ringed by water and wind. The only way to cross the mile-wide Golden Gate Strait was by ferry—slow, expensive, and often halted by fog. The city dreamed of a bridge, but most people felt it would be crazy to try.

The Pacific tides ripped through the Gate at seven knots. The depth below dropped to three hundred feet, and the winds screamed at sixty miles per hour. One engineer famously said it would be easier to bridge the Atlantic.

But Joseph Strauss disagreed.

Strauss wasn't the tallest man in the room, or the most conventionally respected. He was a builder who believed that connection was worth engineering for. He spent years revising his design and knocking on doors that mostly stayed closed. The U.S. War Department objected. Ferry companies, whose profits depended on the waterway, fought bitterly against him. Even local papers mocked his plan as "folly in steel."

With the Great Depression looming, a project that would cost $35 million seemed unthinkable. The city had no cash to spare, banks were failing, and unemployment stretched across California.

Then came a bold idea: six northern California counties would back a bond issue, pledging their property taxes as collateral. It was an act of faith.

Most financiers refused to touch it—until A.P. Giannini, founder of Bank of America, stepped in. "This bridge will build itself out of the courage of the people," he said, buying the entire bond issue when no one else would. In November 1930, voters approved the bonds by a wide margin.

In a time of despair, San Francisco chose to believe in itself.

In 1937, the Golden Gate Bridge opened ahead of schedule and under budget. What had once seemed reckless became an icon that is now inseparable from the city itself. Almost a century later, it moves people, goods, and ideas every day. While $35 million seemed expensive for a multi-year project, the bridge now generates about $150 million per year in toll charges—far beyond what its builders could have predicted. It stands as proof that

when complex systems are designed with discipline and care, they don't just solve immediate problems. They shape the future.

That didn't happen because of bravery alone. It happened because thousands of transactions—financial, legal, operational—were designed to work together. Funding decisions, engineering approvals, safety requirements, construction sequencing. Each one executed with discipline. Each one dependent on the others.

That's what a transaction really is.

When you zoom out far enough, transactions stop looking like isolated events and start looking like building blocks. Every organized system—companies, markets, governments, communities—is an accumulation of transactions. When you stack enough transactions together, you get something recognizable.

A business. A school system. A city. A bridge.

Over time, those transactions build the future of our world. But only when they sit on a solid foundation.

Giving Transactions a Home

The most consequential moments in any organization's life are transactions. And yet, the systems supporting them are almost entirely informal.

Spreadsheets emailed between parties. Wiring instructions confirmed over the phone. Approvals made by whoever happened to be available. No shared framework. No verification layer. No record that could be relied on afterward.

And, because transactions cross almost every department, they tend to have no real home.

When something goes wrong—and eventually, something always does—there's no structure to catch it.

This book exists because that problem is solvable. The Secure Transactions Framework was built from years of working inside high-stakes closings across cities, utilities, school districts, hospitals, and capital markets—watching what actually breaks, and designing around it.

The framework stands on its own. What follows is a guide for anyone who wants to use it.

It's written for the people who conduct transactions on behalf of organizations—and who carry the weight of that responsibility every time money moves. Capital markets professionals, investment bankers, and municipal finance advisors who close deals for a

living. Consultants and counsel who sit inside transactions every day without always having a clear framework to hand their clients. CFOs and controllers who own the financial execution inside their organizations. CEOs who understand that their biggest moments of risk—and opportunity—live in how their transactions are structured and executed.

It's for board members who want to understand where institutional risk actually lives: not just in the strategies they approve, but in the execution of the transactions that carry those strategies forward.

And it's written with an eye toward where this work is heading. Transaction coordination—the discipline of preparing, structuring, and securing transactions before they execute—is becoming one of the most consequential functions in any organization that moves significant capital. The people who do that work well are the ones who make it possible for everything else to move forward.

When transaction systems are designed thoughtfully, people can move quickly without gambling. Teams can act without freezing. Organizations can grow without constantly reinventing controls. The work becomes calmer because the foundation is solid.

That's what the Golden Gate Bridge ultimately delivered: Confidence. Confidence that the crossing would hold. Confidence that the investment would last. Confidence that the system was built for more than fair weather.

Transactions deserve the same care.

If you build them with discipline, they stop being potential points of failure and start becoming infrastructure. Infrastructure that carries your organization's intent forward, day after day, even as conditions change.

Transactions—and all the dreams and progress they support—are too important to be left to chance.

Today, the best path to building a better future is to build on the foundation of secure transactions.

Part One

How Modern Transactions Work

CHAPTER 1: The Transaction Breakdown

Money moves in the present tense

One transaction, in one moment, can turn plans into something real.

They're the mechanism that turns decisions into schools, hospitals, housing, and infrastructure. They're how organizations move from intention to outcome. When they work, communities grow, businesses expand, and progress compounds quietly in the background.

When they don't, everything slows down.

Twenty years ago, early on in my career, I got a job in municipal finance. At the time, it was just an entry point. But those years shaped how I see transactions. It wasn't theoretical work—these transactions funded things people could see and use. When a deal stalled or failed, it didn't just show up as a line item. It delayed a project, strained a budget, or put real pressure on a community.

But when transactions worked, the effect was tangible. Entire towns moved forward.

That stayed with me.

Over time, working across cities, utilities, schools, and hospitals, I started to see a pattern. No matter the size of the organization or the sophistication of the team, transactions carried an outsized amount of risk relative to the attention they received. Months of careful planning could be undone during execution. Not because the strategy was wrong, but because the structure supporting the transaction wasn't designed for how work actually happens.

That pattern shows up everywhere once you know to look for it.

Sometimes it's subtle. And sometimes, like in the case of White Lake Township, it's very expensive.

In November 2024, White Lake Township, Michigan, was preparing to close on a major infrastructure bond. The funding would support a new civic center, and the deal had taken months to put together. For the township, this was a milestone, the kind of transaction a community can point to as progress.

Lawyers had reviewed every detail. Advisors had aligned the terms. Banks and paying agents had worked through all the logistics. Everyone involved had done their job.

Someone else had been doing their job, too.

The attackers hadn't rushed in at the last minute. They had watched quietly for weeks. They learned how the team communicated, when messages were sent, which phrases appeared in routine updates. By the time they acted, they understood the transaction almost as well as the people running it.

Their move was precise and simple: One email, placed at exactly the right moment, with updated wiring instructions. The email address differed from the legitimate one by a single character—a capital "i" swapped for a lowercase "l". Everything else was identical. The sender name, formatting, timing, tone. Nothing stood out because nothing was supposed to.

If you've ever been involved in a closing like this, you know what those final hours look like. Instructions are flowing. Everyone is watching the clock. A team that processes transactions every day does what it always does—follows the paper in front of them. Someone wired $29 million to the account the instructions specified. The account that looked exactly right.

Within hours, the funds meant for the township's civic center had been dispersed out of that account and were gone. Federal authorities moved quickly, and most of the money was recovered. But roughly $5 million was lost, and the consequences of that closing are still being worked through.

When investigators reviewed what happened, they didn't find a software bug or a technical failure. The systems did exactly what they were designed to do. So did the people. The staff member who wired the money wasn't negligent. They were doing what you do when you process hundreds of transactions—send money to the account the instructions tell you to. That's the job. That's what professionalism looks like in a high-volume environment.

The breach wasn't a failure of attention. It was a failure of structure. When billions of dollars move daily across emailed wire instructions, every transaction becomes a potential target. It only takes one weak link. The attacker didn't need to defeat the whole system. They just needed one correctly timed email in a busy inbox.

White Lake Township wasn't a uniquely vulnerable organization. It was a normal one running a transaction the way most organizations still run them today. That's what makes it worth paying attention to. This wasn't an isolated incident or an edge case. It was the first major public display of something that is happening at a rapid pace, in every sector, to organizations that believe their process is good enough because it always has been.

This is the pattern I've seen repeatedly in wire fraud cases—not because people are careless, but because transactions sit in an uncomfortable space. They're complex enough

to involve many participants, but familiar enough that they often feel routine. Risk is assumed to have been addressed earlier, during planning or approval, even though the most irreversible part of the process is still ahead. This is the gap that social engineering takes advantage of.

Social engineering is what happens when someone manipulates the human side of a transaction—its approvals, instructions, or relationships—so the system processes something that shouldn't have been authorized.

You may have seen the effects of this in your own work.

A county loses bond proceeds to a fraudulent account. A law firm wires out a settlement that never reaches the client. A university sends a construction payment to criminals instead of contractors. A healthcare system pays invoices to vendors who never sent them.

Different organizations. Same underlying problem.

Transactions do an enormous amount of real-world work. But the systems supporting them haven't kept pace with how much is riding on execution.

And that's where things start to break down.

Here's the good news: If we can identify and understand the specific areas where transactions are likely to break down, we can make them more secure. And when transactions are secure, all the progress we're building toward stays on track.

Where Transactions Break Down

When I sat down to write this book, I spent a lot of time reviewing some of the most painful fraud cases I've seen over the past couple of decades—along with the investigations, insurance claims, and post-mortems that followed them. I also took a hard look at my own experiences in public finance. After twenty years, you start to notice the same weak points showing up in completely different organizations.

What I found wasn't exotic threats or rare edge cases. It was three areas where transactions are consistently weaker than people realize. Areas where good decisions, good people, and good intentions are undermined by lack of structure. If you work in finance, you've probably brushed up against at least one of them. If you've been through a large, multi-party transaction, you might've even seen all three.

Those areas are **Decisions**, **Governance**, and **Integrity**.

Let's walk through them.

Weak point #1: Making decisions when context is thin. Most transaction problems start earlier than people expect. They begin when a transaction moves forward without a shared understanding of what kind of transaction it actually is.

In practice, many transactions get treated as routine because they resemble something familiar. The amount looks similar. The process feels the same. The people involved haven't changed. So the assumptions from the last transaction quietly carry over.

Which is fine—until something goes wrong.

Some transactions are simple and repeatable. Others are rare, time-sensitive, or irreversible. Some stand alone. Others unlock a chain of downstream commitments. When those differences aren't surfaced early, transactions inherit controls that don't match the real risk.

You see this play out in small ways all the time.

A payment gets approved because it matches prior activity, even though one detail has changed. A disbursement moves forward because it's "part of the process," even though it carries consequences beyond the immediate transfer. A decision gets deferred because revisiting it would slow things down.

Everyone involved is acting reasonably, based on the slice of the transaction they can see.

The challenge is that context is fragmented. One person understands the purpose. Another understands timing. Someone else understands the mechanics. Rarely does that full picture come together before execution begins.

When transactions aren't clearly classified and connected early, decisions drift toward execution. Momentum replaces deliberation. And risk only becomes visible at the point where there's very little room left to respond.

Weak point #2: Enforcing governance when familiarity replaces clarity. Modern transactions move across more people and organizations than most governance structures were designed for.

Internal teams, external advisors, banks, agents, and vendors all play legitimate roles. Each one has responsibilities. Each one does their part. What's often missing is a clear picture of how authority and accountability fit together at the transaction level.

Inside organizations, authority tends to follow titles and departments. Outside, relationships are governed by contracts and long-standing trust. Transactions cut across both.

That's where gaps form.

You see it when approvals are granted based on summaries instead of full context. When execution is delegated without clarity on who owns the outcome. When external partners are deeply embedded in workflows but sit just outside formal controls.

Over time, familiarity fills in for structure.

A known name becomes reassurance. A trusted relationship shortens a step. A long history replaces explicit verification. None of this feels reckless. It feels efficient. It feels like collaboration.

But transactions don't benefit from ambiguity. When responsibility and authority aren't clearly aligned around the transaction itself, assumptions pile up. It's easy to think that if someone is in the transaction, it's because they're supposed to be there (even when they're not).

That space—between organizations, roles, and expectations—is where problems take root.

Weak point #3: Building integrity when instructions move faster than meaning. Most payment systems are very good at recognizing information. They handle account numbers, formats, and routing details with speed and precision.

In day-to-day work, that recognition often feels sufficient. Familiar names, familiar templates, familiar processes all signal that things are on track. As long as nothing stands out, execution continues based on trust.

And trust, when left unverified, becomes a vulnerability.

The challenge is that information rarely travels with full context. Instructions are separated from purpose. Changes are detached from original intent. Sensitive details live across email threads, shared drives, attachments, and screenshots. Over time, identity, authorization, and execution drift into different places.

You see this when wiring instructions arrive separately from approvals. When documentation lives in one system and execution happens in another. When verification is assumed to have happened earlier, but no one can quite point to where.

By the time questions surface, the transaction is often already complete.

Speed compounds this. Irreversible payment rails are used by default. The window to detect and recover from an error gets smaller, even as transaction complexity increases.

Integrity depends on keeping identity, information, verification, and execution connected. When those elements move together, transactions remain understandable. When they separate, small changes become hard to spot until it's too late.

These three weak points—decisions, governance, and integrity—rarely appear in isolation.

In one simple deal, the transaction may not be fully understood at the outset. Responsibility may be spread across multiple parties. Verification could rely on recognition rather than confirmation.

Individually, each issue seems manageable. Together, they create a system that looks stable right up until the moment it isn't.

That's why transaction failures often feel sudden. The warning signs were present, but they were distributed across decisions, governance, and integrity. That dispersion makes them harder to spot.

The purpose of the framework that follows isn't to add friction or slow work down. It's to bring clarity forward—so transactions are easier to understand, easier to manage, and easier to execute with confidence.

Awareness Isn't Enough

Over the last decade, regulators have been sounding the alarm. The FBI continues to report record-breaking losses from business email compromise and socially engineered fraud. Banks have watched their "commercially reasonable security procedures" challenged—not because the bank was hacked, but because attackers changed tactics.

They stopped attacking systems. They started attacking workflows.

And here's the part that surprises people outside finance: most professionals already know this.

You've read the warnings. You've taken the training. You've heard the stories—often from organizations that look a lot like yours.

And yes, that awareness helps. But it isn't reliable under pressure.

Information doesn't save you in the heat of the moment. Not when the project manager wants the payment out before Monday. Not when the inbox is stacked and you're triaging. Not when the request looks normal, comes from the right person, and lands at exactly the right time. In those moments, knowledge doesn't save you. Structure does.

That's why I believe—after watching hundreds of these incidents—that the conversation about fraud has been missing a key element. We keep trying to solve it through education and improved technology, but both fall apart if the underlying **structure** is weak.

And the heart of good structure? Organization.

We need clarity about how decisions get made. We need governance that defines responsibility across internal and external participants. We need integrity built into how information, identity, and verification travel through a transaction.

Because in high-pressure moments, systems outperform the best of intentions.

The Secure Transactions Framework

Once you understand where transactions actually break down, the solutions become a lot clearer.

As a kid, I was obsessed with math because it was something that could be logically figured out. I guess you could say that I've always found comfort in systems that behave predictably.

After years of watching good organizations struggle with the same execution problems, I set out to design a framework that treated transactions as systems—not isolated payments.

Something that would work for a state treasury managing billions, a small municipal office trying to get a project across the line, or anyone else who moves money. Something that didn't rely on perfect attention under stress.

Rather than asking people to be more careful, or asking technology to compensate for weak process, the Secure Transactions Framework starts with a different question:

What would transactions look like if execution was designed intentionally, instead of inherited?

The Secure Transactions Framework isn't a checklist or a single tool. It's a way of organizing how transactions move—so clarity exists *before* money ever leaves an account.

The framework is built around ten core components:

1. **Classification:** Not all transactions carry the same risk, and treating them as if they do creates blind spots. Classification establishes a shared understanding of what kind of transaction this is—routine or exceptional, reversible or irreversible—so safeguards match the real stakes from the start.
2. **Connected Transactions:** Many transactions don't stand alone. They unlock future obligations, depend on earlier approvals, or affect downstream payments. Making

those connections visible prevents small actions from creating unexpected consequences elsewhere.

3. **Transaction Stages:** Transactions unfold in phases, from intent and authorization to execution and record-keeping. When those stages blur, momentum can outrun verification. Clear stages create natural checkpoints without slowing work down.
4. **Organizations:** Authority inside an organization doesn't always line up with how transactions actually move. This component focuses on aligning internal structure, permissions, and accountability so responsibility follows the transaction, not just the org chart.
5. **Contacts:** Transactions rely on external relationships — advisors, banks, agents, vendors — long before money moves. Managing those relationships intentionally reduces reliance on familiarity and ensures external participants fit cleanly into the transaction structure.
6. **Participants:** Every transaction has specific roles: who proposes, who approves, who executes, and who records. Defining those roles at the transaction level prevents authority from drifting and removes ambiguity attackers exploit.
7. **Verification:** Recognition isn't confirmation. Verification establishes clear methods for confirming identity, intent, and changes before execution, so decisions are based on proof rather than assumption.
8. **Information:** Transaction information spreads easily across systems that weren't designed to preserve context. This component focuses on keeping instructions, approvals, and sensitive data connected to meaning throughout the transaction lifecycle.
9. **Insurance:** Insurance limits damage, but it only works when structure comes first. Designing transactions with evidence and process clarity ensures coverage is meaningful and recoveries don't depend on reconstruction after loss.
10. **Payment Rails:** *How* money moves matters as much as *why*. Different rails offer different levels of speed, reversibility, and recovery. Matching the rail to the transaction's risk preserves options when something goes wrong.

Each of these addresses a specific failure point we've already discussed. Together, they move security out of training sessions and into the structure of execution itself.

You don't need to be perfect to avoid fraud. And you don't need to operate in constant alert mode. All you need is to design your transactions intentionally.

Transactions are the heartbeat of an organization, and of the world that gets built and shaped around us. Every movement of money carries trust, value, and reputation in a single click. Making that click secure is what allows projects to move forward and plans to become reality.

In the chapters that follow, we'll walk through each part of the framework in detail—using real transactions, real failures, and real recoveries.

But to move forward, we first need to understand how we got here.

CHAPTER 2: The Great Disconnection

Fragmentation hides what connection reveals

If you go back to the early 1800s, the way money moved across America looked nothing like what we know today. If you wanted to send funds from New York to Cleveland, you didn't hit a button: you hired someone. And the money traveled exactly as fast as that person could ride or walk there. Two weeks, if you were lucky.

Banking wasn't a national system yet. It was a patchwork of small, local relationships. People didn't choose their bank based on interest rates or online tools. They chose the one down the street because that's where their father and grandfather had banked.

Your banker didn't just know your business. He knew your character, your family, your reputation. If you asked for credit, he didn't run a credit score. He thought about whether you'd kept your word in the past and whether your neighbors respected you.

Everything ran on trust that built over time, the way real relationships do.

These early banks were chartered by state legislatures, but the process was messy—and more political than regulatory. Whoever was in power granted charters to the people who supported him.

But once established, banks couldn't afford to be reckless.

If a banker mismanaged deposits, he couldn't hide behind a corporate office in another state. He would lose his standing in a community where everyone knew his name, and his family. Accountability was immediate, and it was personal. You didn't defraud people you'd see at church the next Sunday.

Funny enough, the biggest security feature of the system was something we don't usually think of as a feature at all: **distance**.

Banking was profoundly local because it needed to be. And even though that probably felt like a limitation at the time, it actually created security. Money couldn't outrun horses or sail faster than the wind. Transactions unfolded slowly enough that everyone involved could double-check what mattered. There wasn't much room for deception because everyone saw each other, talked to each other, and lived close enough for word to spread if something smelled off.

This was especially clear in one of the biggest government-funded projects of that time.

It started in 1808, when Jesse Hawley sat in debtor's prison in Canandaigua, New York, staring at the walls that represented his failed flour business. He couldn't afford to transport his goods to market. The Appalachian Mountains separated the western frontier from the eastern seaboard, and moving a ton of goods from the Great Lakes to New York City cost $100 and took three weeks by wagon—when the roads were passable at all.

From his cell, Hawley wrote a series of essays proposing something completely unheard of: a man-made waterway stretching 363 miles from Albany on the Hudson River to Buffalo on Lake Erie.

Critics said he was crazy. President Madison and his successors refused to support the idea with federal funding because they said internal improvements should be funded by the state.

But New York's governor, DeWitt Clinton, saw this incredible vision as a chance to turn New York into the commercial center of the country.

In 1817, the New York State Legislature approved $7 million in state bonds (roughly $150 million today) to fund the canal entirely on its own. No federal help. No safety net. The money came through the newly created Canal Fund, and would be managed by state commissioners.

It was a completely new idea: the canal would pay for itself through toll revenue. This was America's first true infrastructure revenue bond model—a direct ancestor of today's municipal finance structures.

And here's where the story gets interesting for us in finance.

The oversight was local, immediate, and personal.

Commissioners lived close enough to the work to hear if a contractor was cutting corners. If something looked off, someone would literally ride out to inspect it. Reports were published. Money was traceable. Everyone involved had a reputation on the line.

Construction began on July 4, 1817. There were no civil engineers then. No computers. No offices full of analysts. Just surveyors, stonemasons, and laborers using hand tools, horses, and black powder.

They carved a route through dense forest, rocky cliffs, and mosquito-infested swamps. The design included 83 locks to lift boats 568 feet from the Hudson River to Lake Erie. Each lock was hand-laid stone and timber, sealed with hydraulic cement.

The project was not without its issues. Thousands of workers died of malaria, accidents, and exhaustion. Yet the work pressed on, section by section, with remarkable speed and precision. The system was slow by modern standards, but it was accountable.

By 1825, the Erie Canal was complete. Governor Clinton sailed from Buffalo to New York City and poured water from Lake Erie into the Atlantic—a symbolic "wedding of the waters."

The payoff was immediate. Shipping costs between Buffalo and New York dropped from $100 to $10. Travel time went from three weeks to six days. The state repaid the entire $7 million debt in just nine years. By the 1830s, tolls were producing millions in surplus revenue for the state treasury.

But the part I find the most striking—and the most relevant to the world we live in now—is this: Everyone involved could see just enough of the whole system to know whether things were going right.

The commissioners understood the engineering. The bankers understood the financing. The contractors understood their responsibilities. The legislators understood the public trust they were protecting.

No one had a perfect view of everything. But the overlap between each person's understanding created something today's system doesn't provide: shared visibility.

And that visibility made fraud, waste, and large-scale mistakes incredibly hard to hide.

That period—the early era of American finance—was the height of relationship-driven banking. The money moved slowly, but it moved with certainty. People trusted the system because they trusted each other.

Then came the technology that would change this model forever.

The Technology-Relationship Hybrid

On May 24, 1844, Samuel Morse sent the first long-distance telegraph message from Washington to Baltimore: "What hath God wrought?"

Of course, the telegraph didn't immediately revolutionize banking. At first, it was just a strange new tool—wires hung along railroad tracks, buzzing with messages most people never saw. But it was the beginning of something that would quietly reshape how money moved.

By the 1850s, telegraph lines stretched along the East Coast and across the continent. At first, railroads saw the wires strung across their tracks as a nuisance. Then, they started using them to coordinate traffic.

Before long, those same wires would become financial arteries.

For the first time in history, information could move faster than people. The telegraph separated communication from transportation. News that once took two weeks to travel from New York to Cleveland now arrived almost instantly. According to business historian H.W. Brands, this gave financial markets in New York a huge advantage. Someone watching with access to the fastest information could dominate the nation's economy. Suddenly, being able to access information was more valuable than being physically present.

And then, in 1872, Western Union introduced a money transfer service.

It worked like this: A customer handed cash to a telegraph operator in one city. That operator sent a coded message—using passwords and reference books—to an operator somewhere else. That operator released funds to the recipient.

Money stayed put physically, but the authorization could leap across thousands of miles in minutes. By 1877, Western Union was moving nearly $2.5 million annually.

If you were a business owner in 1920—say, a textile manufacturer in Massachusetts—you lived in this hybrid world. You still walked into a local bank where the teller knew your family and the banker understood your seasonal cash flows. But when you needed to pay a cotton supplier in Georgia, something remarkable happened behind the scenes.

Your banker might pick up the phone and call a banker in Atlanta—a man he'd known for years. Someone he'd met at conferences, negotiated deals with, and probably shared whiskey with. That Atlanta banker knew your supplier the same way your banker knew you.

So when money moved, context moved with it.

Your banker knew whether your request made sense. The Atlanta banker knew whether the supplier was expecting funds. Any unusual detail—a strange amount, different timing—raised questions immediately.

This system blended speed with trust. Technology transmitted the message, but relationships still verified the meaning.

It wasn't perfect—nothing is—but it scaled surprisingly well. Banks could now serve customers across huge distances without losing the natural guardrails that came from

personal knowledge. Fraud had a hard time sneaking in because too many people along the way understood the full story.

And this is the part I think resonates today. Speed didn't break security—as long as people stayed connected.

The problem is, that connection didn't last.

As technology got faster and banking grew larger, the human layer—the informal conversations, the shared history, the gut checks between bankers who actually knew each other—started to fade into the background.

The telegraph didn't destroy the relationship model. It just planted the seed for a world where information could outrun context.

Then everything broke.

How Relationships Got Disconnected From Finance

When the stock market crashed in 1929, it wasn't just the markets that fell apart—trust did too.

People lined up outside banks hoping their deposits were still there. Many discovered they weren't. Commercial banks had been mixing deposit accounts with risky securities speculation. And when those bets failed, customer money vanished with them.

Senator Carter Glass believed the problem wasn't bad bankers—it was bad structure. He decided to cut those two types of banking in half. That way, they could never put depositors' money at risk.

So in 1933, Congress passed the Banking Act, and forced every institution to make a choice: You could be a commercial bank. Or you could be an investment bank. But you could no longer be both.

Banks had one year to decide.

On paper, it was a clarity move designed to protect depositors—and that immediate goal succeeded. Deposits became safer. Banks stopped gambling with money that belonged to farmers and factory workers and families trying to save for their futures.

But this structure introduced a tradeoff that wasn't immediately obvious.

Banks that had once understood their customers from top to bottom suddenly split into narrow specialities. A business owner who used to rely on one banker for loans, deposits, and investments now had to deal with multiple institutions. Each one of these only saw a slice of his financial life. Nobody had the full picture.

That fragmentation started as a safety measure. But forty years later, technology arrived and took it into overdrive.

In 1973, the Society for Worldwide Interbank Financial Telecommunication—SWIFT, for short—launched a standardized messaging network that completely reshaped global finance. It was great for efficiency. Banks could now send structured, machine-readable instructions between banks instantly. No more deciphering telex messages or handwriting. No more delays.

But speed had an unintended consequence: banks stopped calling each other.

The informal checks—"Hey, does this look right to you?" "Is your client expecting these funds?"—disappeared.

A SWIFT message looked legitimate whether it came from a trusted partner or a criminal who had studied the formatting. Banks processed instructions based on syntax, not relationships. The system worked flawlessly for moving money, but it stripped away the context that had once helped people spot red flags.

By the 1980s, the shift was complete. Transactions now passed through:

- An originating bank
- Possibly one or more corresponding banks
- A receiving bank

Each institution in the chain did its part correctly. A bank might validate the formatting. A correspondent might route the payment. A receiving institution might credit the account.

But no one knew whether the payment made sense.

If something unusual happened—like a sudden change in beneficiary, a new account number, or a large transfer that didn't match historical patterns—the system had no memory of the past relationship to compare it against.

The question of, "Does this fit what we know about this customer?" had disappeared.

When visibility shrinks, deception thrives. When every participant only sees their tiny slice, it's like the whole picture stops existing in anyone's mind.

By the end of the 20th century, money could move around the world in seconds. But the people moving it had lost the ability to see whether the request came from a familiar partner, or someone pretending to be one.

The Massachusetts textile manufacturer of 1920 operated in a world where his banker knew his reputation, his business, and his relationships. His modern equivalent interacts with systems that know his routing number, but not his story. The Erie Canal commissioners could walk the route, talk to contractors, and verify work with their own eyes. Today's infrastructure projects involve hundreds of participants spread across jurisdictions, none of whom see more than a portion of the money flow.

The disconnection between how money moves and who understands it laid the groundwork for exactly the kind of fraud we see today.

How Money Moves Today

In early 2016, at Bangladesh Bank, investigators began tracing a series of fraudulent payment instructions that had been sent through the global banking system. Nearly a billion dollars had been requested. Most of the transfers were stopped.

Five weren't.

By the time the picture came together, $81 million had moved through New York correspondent accounts, into Manila, and then into places where it was effectively unrecoverable.

What stood out wasn't a technical breakdown. The messages were valid. The formatting was correct. Every system involved processed what it was given. Each institution followed its procedures. From the outside, everything looked alright.

The problem was in the way money moves.

Modern payments don't move money in the way most people picture it. No funds physically travel from one place to another. What moves are records—ledger entries updated across different institutions, under different regulators, on different schedules. A single payment becomes a sequence of handoffs. Each participant sees its part clearly. No one sees the whole thing at once.

As that chain stretches across time zones and settlement windows, visibility thins. A payment can be debited in one place while it's still waiting to be credited somewhere else. Overnight and over weekends, transactions sit in a sort of twilight zone—accounted for on one side, unresolved on another.

That's the environment the Bangladesh Bank attackers understood. They didn't outsmart the system. They worked within it. They knew that once a payment left the originating account, responsibility would fragment. Each institution could confirm what it received. No one could confirm what the payment meant in context.

This is the modern execution gap. Not because people aren't careful, but because the structure assumes that certainty survives the handoff. It doesn't. Once money leaves the account, it enters a system designed to move quickly, not to explain itself. And by then, the window to confirm what's real has already closed.

Here's the simple takeaway.

The system didn't fail. It evolved. And as it evolved, some of the visibility that once came naturally became harder to maintain. Money learned how to move faster and farther than ever before, while the context that once surrounded it grew thinner with each handoff.

Once you understand how money moves today, execution becomes easier to see clearly. You can trace where certainty fades as transactions pass from one participant to the next. You can also identify the specific moments where clarity still exists, even if only briefly.

That matters, because it points to the real challenge. Modern transactions are structured in a way that assumes understanding survives the handoff from one party to another. In practice, that assumption often doesn't hold.

Getting transactions right depends on being deliberate about where understanding is strongest, and intentional about how decisions, authority, and information move forward once that understanding begins to fragment. Structure has to carry what experience used to handle informally.

This creates an opportunity. When execution is designed with that reality in mind, transactions become calmer, more predictable, and easier to manage under pressure.

Before we talk about frameworks or solutions, there's a more basic shift that needs to happen.

We need a clearer way of thinking about transactions themselves—how they unfold over time, where certainty exists, and what has to be in place before money ever moves.

That's where we'll go next.

CHAPTER 3: Framing the Framework

Money only becomes dangerous once it starts moving

Most people think of transactions as moments.

A wire is sent. A payment clears. A contract is signed. That's the part that feels decisive—the point where something happens and money moves. But those moments are endpoints, not events.

Long before money moves, a series of quieter steps has already taken place. Someone decided what needed to happen. Someone determined who should be involved. Assumptions were made about timing, authority, and responsibility. Information was shared, copied, and forwarded. Approvals were granted, explicitly or implicitly.

By the time a transaction reaches execution, most of the meaningful decisions are already behind it.

You can see this clearly in everyday work.

Take a large vendor payment. The wire itself takes minutes. The work that makes it possible often starts weeks earlier—selecting the vendor, agreeing on terms, aligning approvals, deciding how and when funds will be released. The payment feels like the transaction, but it's really the final step in a longer sequence.

When that sequence is clear, execution feels straightforward. When it isn't, even simple transactions feel fragile.

This is where many organizations get tripped up.

Attention stays focused on execution, even though execution is the point with the least flexibility and the highest consequences. Risk shows up late, when there's little room to adjust, because the order of decisions wasn't fully established earlier.

Transactions don't happen all at once. They unfold. Certain things have to be true before others can safely occur. When that order is unclear, small gaps turn into real problems.

This chapter is about making that order visible.

The Distinction Between Transactions and Payments

Money movement isn't a single action. It's a system. And like most systems, it has a hierarchy.

At the top of the hierarchy is the **transaction**. This is the container. It defines the scope, the participants, the rules, and the lifecycle. A transaction might last a day, a year, or decades. It establishes what success looks like and when the work is actually finished.

Inside the transaction sits the **flow of funds**. This is the design. It describes how money is meant to move over time: the order of events, the conditions that unlock each step, and what happens when something changes. A single transaction may have one flow of funds or many, running sequentially or in parallel.

At the bottom of the hierarchy are **payments**. Payments are the individual executions—wires, ACH transfers, checks, internal movements. They are precise and important, but they are not self-explanatory. A payment without context is just motion.

When organizations focus only on payments, they're working at the lowest level of the hierarchy. They're managing *execution* without managing *intent*.

I like to think of this hierarchy through the lens of cooking.

A transaction is the cookbook: it defines what you're trying to make, and the boundaries you're operating within. A cookbook might contain a single recipe or hundreds. It might be fixed, or it might evolve over time. But without the cookbook, nothing is repeatable. Every meal becomes an improvisation.

The flow of funds is like the recipe. Recipes describe sequence and dependency. What happens first, what happens next, what can run in parallel, and what must wait. A recipe tells you not just *what* ingredients to use, but *when* and *how*. Miss the order, and the outcome changes—even if all the ingredients are correct.

Lastly, payments are the ingredients. They're the tangible things you move around. Flour, salt, oil. Wires, ACH, checks. Ingredients matter, but ingredients alone don't make a meal. Spread them across the counter without a recipe, and nothing useful happens.

This is where many financial systems struggle. They're very good at tracking ingredients. They know where the money went. They know when it moved. But many times, they don't preserve the recipe—or even acknowledge the cookbook exists.

Here's why this distinction matters: You can have a cookbook with one recipe and two ingredients. Or, you can have a family cookbook that grows over generations—adding new recipes, reusing familiar ingredients, and improving outcomes over time.

When the cookbook is well organized and shared, others can reproduce the result. They can move faster because they don't have to rediscover the process. And they can even improve it because they understand how the pieces fit together.

But when the cookbook isn't organized—or isn't shared—everything becomes fragile. Knowledge lives in someone's head. Exceptions pile up. And speed starts to create risk instead of efficiency.

The same thing happens with money. Teams that organize around transactions create clarity. Teams that organize around payments create noise. And no amount of checking ingredients can replace a missing recipe.

For most people, this hierarchy feels obvious once you see it. So, why do so many capable organizations still struggle to distinguish payments from transactions?

Because financial systems weren't built to support that distinction.

Most financial systems are built from the bottom up. They start with the payment because that's the visible action. A wire gets sent. An ACH clears. A balance changes. Everything else is layered on afterward.

The problem? There's no place to keep track of intent. Accounting systems are designed to capture what already happened. Banking systems are designed to move money reliably from one account to another. Software evolved to mirror those workflows. Over time, payments became the unit everything revolved around—because payments were the most concrete thing the systems could see.

That payment-first view is reinforced everywhere. Banks talk about transactions when they mean transfers. Software logs payments without preserving why they existed. Teams are rewarded for execution—getting wires out, clearing exceptions, closing books—not for organizing context upstream.

When tools are designed around payments, structure has to live somewhere else—usually in emails, spreadsheets, or someone's head. As volume increases or people change roles, the gaps widen. More checks get added. More approvals pile on. Effort increases, but speed still drops.

The root problem? There's a mismatch between how money actually moves and how systems were built to observe it.

Hierarchy may feel unfamiliar because most systems never made room for it. Once you see that, the problem shifts. It's no longer about trying harder at execution. It's about reorganizing where understanding lives.

How to Design for Intent, Not Just Movement

In my experience, most organizations don't need more controls or more people to improve how money moves. They need to reorganize where decisions happen.

When you start with the payment, you sacrifice context. Every movement of money feels like a fresh decision. Approvals get repeated. Verifications happen late. Risk gets managed under deadline pressure. Security turns into something you bolt on instead of something the system provides.

The result is something we've all seen before. Teams rely on experience and memory to fill in gaps the system doesn't see. Speed depends on who's in the room. When those people change, the process slows or breaks.

Here's the shift: **Start organizing around transactions**.

A transaction is where intent lives. It's where purpose, scope, and expectations are defined once—so they don't have to be rediscovered at every step.

Organizing around transactions means deciding *what you're doing* before you decide *how to do it*. It means naming the objective, identifying who's involved, and making the lifecycle visible up front.

When you do that, something important happens. Decision-making improves, and security improves with it. You didn't even need to add more checks because fewer decisions are left to interpretation. Fewer assumptions get made under pressure.

Intent and purpose aren't just conceptual. They're operational. When they're explicit, systems can enforce them. When they're implicit, people have to remember them—and that's where things break down.

So, make intent explicit before money flows.

Disagreements aren't often the main point of friction in finance. In my opinion, ambiguity is a much bigger problem. When intent isn't clear, people fill in the blanks differently. Controls feel arbitrary. Security feels more suffocating than supportive.

Here's the good news: Making intent explicit doesn't require long documentation. All you need to do is answer a simple set of questions before money moves:

- What is this transaction trying to accomplish?
- Over what period of time?
- Who is involved, and why?
- What does "complete" actually mean?

Once those answers exist at a transactional level, downstream work gets easier. Teams aren't guessing why a payment exists. They're executing against a known purpose.

In practice, adapting to the hierarchy of transactions looks like this:

- **First, define the transaction.** Name it. Describe its purpose. Identify its lifecycle. This is where intent belongs, and it should happen once.
- **Second, design the flow of funds.** Decide how money should move over time. In what order. With what conditions. This is where structure replaces improvisation.
- **Third, execute payments.** Wires, ACH, transfers, checks—these become execution steps, not moments of decision-making.

One of the most important things this hierarchy does is reduce the number of decisions that have to be made under pressure.

When transactions are defined clearly, many decisions get made once instead of repeatedly. Purpose is established. Boundaries are set. The flow of funds reflects intent. By the time a payment is ready to execute, there's very little left to debate.

That compression matters more than most people realize.

Teams slow down not because money moves too fast, but because too many decisions are deferred to the moment of execution. Questions that should have been answered days or months earlier get asked when deadlines are tight. People are forced to interpret intent instead of applying it.

Hierarchy changes that. It moves decision-making upstream, where there's time to think clearly. Execution becomes calmer because fewer judgments remain. Approvals become lighter because they're reinforcing known structure, not reconstructing it.

The effect is subtle but powerful. Meetings get shorter. Exceptions become obvious. Teams spend less time re-explaining and more time moving forward. Speed increases because uncertainty drops.

Instead of asking, "Can we send this payment?" the system is already answering, "This payment is part of a defined transaction, following an approved flow."

The Right Structure Enables Speed

Speed has a reputation problem.

In many organizations, speed gets associated with urgency—moving quickly, cutting corners, pushing work through before something stalls. Over time, that association makes speed feel risky, especially when money is involved.

But in the systems that operate the fastest and carry the highest stakes, speed shows up differently.

Air traffic control works because everyone agrees on altitude, routing, and handoffs before a plane ever leaves the ground. Emergency medicine works because teams organize roles, protocols, and sequences in advance. Professional kitchens serve hundreds of meals an hour because stations, recipes, and order of operations are already defined.

In each of these environments, people move quickly because they understand the system they're operating inside. They know where decisions belong, how actions relate to one another, and what has to happen first. Guesswork drops away, and coordination takes its place.

Money movement works the same way.

When transactions feel slow, fragile, or stressful, it usually traces back to missing structure. Decisions surface late. Context lives in scattered places. Execution carries questions that should have been settled earlier.

When structure is clear, execution feels lighter. Fewer last-minute explanations are needed. Fewer handoffs depend on memory or trust alone. Work moves forward without constant pauses to reconstruct intent.

That's what hierarchy makes possible.

By organizing money movement around transactions, flows, and payments—in that order—you move decision-making upstream. Purpose is defined once. Order is established before execution begins. By the time money moves, there's very little left to interpret.

If you only remember one thing from this chapter, let it be this:

- **Transactions** create meaning
- **Flows** create order
- **Payments** create movement

This doesn't eliminate complexity, but it gives complexity a shape. Speed becomes a natural outcome of clarity rather than something that has to be forced under pressure.

That's where the next part of this book begins.

The framework that follows is built to reinforce hierarchy where modern transactions are weakest. It's organized around the three areas that consistently determine whether a transaction holds together or comes apart as it unfolds. As we mentioned in Chapter 1, these are:

- **Decisions:** How transactions are classified, connected, and understood early enough to guide execution
- **Governance:** How authority, responsibility, and roles are aligned across internal teams and external participants
- **Integrity:** How identity, information, verification, and execution stay connected as money moves

Each of these reinforces the hierarchy you've just seen. Together, they're designed to carry clarity forward, even as transactions become more complex and time-sensitive.

Once that structure is in place, speed stops feeling dangerous. Instead, it feels like controlled, steady progress.

Part Two

The Secure Transactions Framework

Decisions

This section focuses on the decisions that happen before money ever moves. You'll learn how to recognize what kind of transaction you're dealing with, understand how individual actions connect to larger outcomes, and structure transactions so intent and risk are clear from the start. When these decisions are made deliberately, everything that follows becomes easier and more reliable.

In this section of the framework, you'll learn how to get clarity on:

- Classification
- Connected Transactions
- Transaction Stages

CHAPTER 4: Classification

Different transactions behave differently—and must be treated differently

---

After the Civil War, the United States needed a unifying project. The country was connected by ambition and telegraph, but not by steel. Crossing the continent by wagon took months and cost lives. Shipping goods by sea around the tip of South America took half a year.

Congress imagined a transcontinental railroad—a single artery binding the Atlantic and Pacific into one economy. The Pacific Railway Act of 1862 authorized two private companies to build it. Union Pacific would start in Omaha and head west. Central Pacific would start in Sacramento and head east.

To accelerate construction, Congress created an incentive structure. The railroad companies would be paid per mile built: more for difficult terrain, less for plains. Land grants sweetened the deal. The faster the companies built, the more they earned.

On paper, the system worked. Track was laid at a remarkable pace and the rails finally met at Promontory Summit in 1869. Newspapers celebrated the achievement.

What Congress didn't fully account for was how they had classified the work.

Every contractor was treated the same way. The project was categorized as routine construction, paid on volume, even though it was unprecedented public infrastructure with enormous complexity and no historical precedent.

That classification decision mattered.

Thomas C. Durant, a financier and vice president of Union Pacific, recognized the vulnerability immediately. He and a group of insiders formed a separate construction company—Crédit Mobilier of America—which Union Pacific then hired to build the railroad at inflated prices. Because the same people controlled both sides, they were effectively paying themselves with federal funds.

The system did exactly what it was designed to do. Miles were built. Payments were made. No alarms went off. The problem wasn't execution. The problem was that an unprecedented, high-risk public project had been treated as routine contracting.

Years later, investigations revealed massive corruption. Millions had been siphoned off. Public trust was shattered. And yet, the railroad itself remained. The physical achievement stood, even as the financial structure beneath it was exposed as deeply flawed.

The lesson isn't that Congress failed to monitor the project closely enough. It's that they misclassified it from the beginning. By treating something unprecedented as familiar, they applied the wrong rules to the highest-risk movement of money in the country.

This is the danger of misclassification. Not the bureaucratic kind, where someone checks the wrong box on a form. The structural kind, where the design of the transaction itself creates opportunities for exploitation.

Fast forward 150 years. Cabarrus County, North Carolina, 2018. Criminals impersonating a construction vendor sent perfectly formatted invoices and new bank details to the county finance department. The county had paid the same vendor before, so when the next batch of invoices arrived, staff assumed the process was routine. What they didn't see was that these payments—three of them, rapid and irregular—were one-time disbursements disguised as ongoing work.

The county treated something unprecedented as familiar. Finance processed the payments using the same verification procedures they used for regular vendor payments. By the time anyone questioned the pattern, $1.7 million had been wired to criminals overseas. Nothing in the system had broken. What went wrong was conceptual: irregular, high-risk payments were treated as part of an ongoing relationship.

In both cases, the system worked as designed. The design was wrong because the transaction type was wrong.

Why Classification Comes First

Classification is the act of naming what kind of transaction you're dealing with before money moves.

At the transaction level, classification answers a simple but critical question: What is the nature of this money movement?

This isn't semantics. Different transaction types behave differently under stress. They fail in different ways. And they demand different levels of coordination, verification, and structure.

Classification happens at the transaction level—not at the payment level.

Every transaction belongs to a type. And that type determines what happens next:

- How much verification is required
- Who has the authority to approve
- How fast money should move
- What kind of oversight applies

When classification is wrong, controls are misapplied. Low-risk processes get layered onto high-risk activity. Familiarity replaces scrutiny. Responsibility fragments. Risk remains invisible until it's realized.

When transactions are classified correctly, the flow of funds can be designed with intent. Payments become execution, not guesswork. Security becomes a property of the system rather than a burden placed on people.

And only then does money move the way it should—fast, predictable, and safe.

The Four Main Transaction Types

Once you understand hierarchy, classification becomes much simpler. You're no longer trying to guess risk from individual payments. You're identifying what kind of transaction you're dealing with and letting that guide everything else.

At a minimum, every transaction falls into one of four types. Each exists for a reason. Problems come up when they're confused, blended, or treated as interchangeable.

A **one-time transaction** is exactly what it sounds like: a single event with a clear beginning and end.

You use one-time transactions when:

- The parties don't expect to interact again
- The purpose is narrow and specific
- The transaction doesn't need to be repeated
- There are minimal downstream dependencies

Examples include a single purchase, a one-off reimbursement, or a standalone professional fee.

What makes one-time transactions risky isn't the dollar amount. It's the lack of history. There's no pattern to validate against. No established rhythm. Identity assumptions are weaker, and errors are harder to spot because nothing "looks wrong" yet.

That means one-time transactions should trigger:

- Maximum verification up front
- Independent confirmation of identities and accounts
- Explicit authorization tied to the transactions purpose
- Slower execution by design, not by accident

When one-time transactions are treated casually or rushed because they appear small, they quietly become some of the highest-risk movements of money in an organization.

An **ongoing transaction** isn't an event—it's a relationship.

You use ongoing transactions when:

- Money moves repeatedly over time
- The same parties interact again and again
- Patterns and history matter
- The transaction evolves

Vendor payments, payroll, debt service, and subscription arrangements all fall into this category.

The strength of ongoing transactions is efficiency. When they're classified correctly, context accumulates. Identities can be established once and reused. Normal behavior becomes clear, and exceptions stand out immediately.

Ongoing transactions should trigger:

- Streamlined processes for routine payments
- Reuse of verified identities and accounts
- Clear thresholds for what counts as an exception
- Periodic re-verification to prevent familiarity from turning into blindness

The most common mistake is treating an ongoing transaction as a series of unrelated one-time payments. When that happens, context fragments. Controls get duplicated or bypassed. Instead of compounding efficiency, every payment becomes a fresh decision.

A **transfer** describes movement without transformation.

You use transfers when:

- Ownership doesn't change
- There are no conditions to satisfy
- Funds are simply being relocated

Examples include moving money between internal accounts, sweeping funds, or rebalancing liquidity.

Transfers are often perceived as low risk—and when they are truly transfers, they can be. But that perception is exactly what makes them dangerous when misclassified.

Transfers should trigger:

- Clear confirmation that ownership is not changing
- Separation of duties, especially for internal movements
- Explicit verification even when both sides “belong” to the same organization

The moment a transfer involves an external party, timing dependencies, conditional release, or irreversible finality, it stops being a transfer. Treating it as one after that point is how internal trust gets exploited.

A **closing** is a transaction that resolves something.

You use closings when:

- Multiple parties must coordinate
- Conditions must be satisfied before money moves
- Funds are released alongside documents, approvals, or events
- Finality matters

Closings appear in many forms: real estate purchases, mergers and acquisitions, bond issuances, capital raises, and large capital projects.

What makes a closing unique isn’t the size of the payment—it’s the coordination.

Closings should trigger:

- Shared context across all participants
- Verification before release, not after
- Sequenced flows of funds tied to conditions
- A clearly defined point of completion

When closings are treated like large transfers or rushed payments, participants operate in isolation. Assumptions replace verification. Speed increases risk instead of reducing friction.

Closings are where classification matters most—and where misclassification can be the most costly.

Once you classify the transaction correctly, everything downstream gets easier. The flow of funds can be designed intentionally. Payments become execution, not interpretation. And the system starts working with you instead of against you.

Grouping Transactions to See the Full Lifecycle

Classification answers one essential question: *what kind of transaction is this?*

But in practice, that question is rarely enough on its own.

Most real-world money movement doesn't happen as isolated events. It unfolds across time, through multiple transactions, often involving different teams, different systems, and different types of activity. When those transactions are treated independently, context erodes. Decisions that once made sense become harder to evaluate later. People are forced to re-interpret intent instead of relying on it.

That's where grouping comes in.

Grouping is the discipline of recognizing when multiple transactions—often of different types—exist for the same underlying purpose. It doesn't change how those transactions are classified. It simply acknowledges that they belong to a larger whole.

Take the lifecycle of a home purchase, for example:

- A mortgage begins with a **closing**
- The monthly mortgage payments that follow are an **ongoing** transaction
- Moving money between accounts to support those payments is a **transfer**
- An early principal payoff is a **one-time transaction**

Each behaves differently and should be treated differently. But none of them exist in isolation. They all serve the same purpose: acquiring and paying down a home.

When those transactions are grouped together, something important happens. The system doesn't just see movement; it sees meaning. Intent carries forward instead of disappearing after the first event. Decisions become easier because they're made in context, not in a vacuum. The question shifts from "Does this payment look right?" to "Does this make sense as part of the whole?"

Without grouping, organizations tend to relearn the same information over and over again. Every action feels like a new decision because the system has no memory of why the transaction exists. Reviews take longer. Controls get applied inconsistently. Familiar activity becomes invisible while unusual activity blends in.

Grouping doesn't coordinate transactions, and it doesn't manage timing or order. Those problems come later (and we'll talk more about them in the next chapter).

What grouping does is preserve understanding. It keeps related activity connected even when it spans months or years, crosses departments, or uses different payment methods. It allows people and systems to reason about money movement as a lifecycle instead of a series of disconnected steps.

Most importantly, grouping changes how people think. Instead of reacting to individual payments, they start recognizing systems. Instead of relying on memory, they rely on shared structure. Things move forward faster because less has to be re-explained.

What Good Classification and Grouping Actually Buys You

When classification is done well, the work starts to feel easier—and not in a vague, motivational way.

The first thing this changes is how people talk to each other. Teams stop debating process because they're no longer confused about purpose. Everyone understands what kind of transaction they're working on before they debate how to execute it. A closing isn't treated like a transfer. An ongoing relationship isn't handled like a one-off exception. That shared understanding removes a lot of unnecessary back-and-forth.

Efficiency follows almost immediately. When transaction types are clear, systems don't have to treat every movement of money as a brand-new puzzle. Routine activity moves smoothly because it's recognized as routine. Reviews go faster because context already exists. Exceptions stand out because they don't belong. Teams stop over-controlling what's familiar and under-controlling what's unusual.

Speed improves for the same reason. Not because anyone is rushing, but because fewer decisions are left unresolved until the last minute. When people agree on what they're doing, execution stops feeling heavy. Money moves quickly where it should, and deliberately where it needs to. That balance is nearly impossible without classification.

One of the biggest changes is how understandable the work becomes across the organization. Transactions no longer make sense only to the people closest to them. Finance, treasury, legal, operations—everyone can see what's happening without reconstructing the story from scratch. Knowledge travels. Work becomes repeatable. Progress doesn't stall just because someone is out of the office or a role changes hands.

That's what classification really unlocks: organization without rigidity, efficiency without shortcuts, speed without recklessness, and security without fiction.

If you only remember one thing from this chapter, let it be this:

Different transactions behave differently. When you name them correctly, the system knows how to treat them.

Once transactions are classified and grouped, money movement stops feeling like a series of isolated actions. It starts to look like what it really is: a set of related decisions unfolding over time, in service of a clear purpose.

And that clarity sets the stage for the next challenge—what happens when those related transactions must move together, in the right order, without breaking the system that connects them.

CHAPTER 5: Connected Transactions

A transaction is only as strong as its weakest dependency

By the late 1970s, the city of Boston had grown so much that it created a serious problem. The elevated Central Artery—six lanes of rusted steel and gridlock—cut through the city's core. City leaders proposed an ambitious solution: bury the highway underground and reclaim the surface of the city.

The project became known as the Big Dig.

On paper, the idea was a feat of modern engineering, and the project looked well-put-together. The work was classified, funded, and approved as a major infrastructure project. Roles were defined. Contracts were awarded. The transaction itself—federal funding in exchange for construction—was legitimate, authorized, and properly structured.

But even as the crew broke ground in 1991, there were cracks in the workflow that no spreadsheet could capture.

Digging a tunnel under one of America's oldest cities meant working around subway lines, gas mains, historic foundations, and a shifting water table. Hundreds of contractors operated under tightly scoped agreements. Engineers solved individual problems as they appeared. Each firm did its job. Each payment was made according to plan.

What no one owned was the sequence.

The Big Dig wasn't one transaction. It was thousands of connected actions that had to align over time. Design decisions affected construction. Construction affected inspections. Inspections affected releases of funds. Every step depended on others finishing first, finishing correctly, and finishing in the right order.

Those dependencies weren't mapped as a system. They lived in the gaps between scopes of work.

As delays mounted, fixes overlapped with unfinished work. Design changes propagated through active construction. Contractors moved forward based on local confirmation, not system-wide verification. Everyone saw their piece clearly. No one could see how one step unlocked—or compromised—the next.

In July 2006, that visibility gap had a tragic outcome. A twelve-ton ceiling panel collapsed in a tunnel, killing Boston resident Milena Del Valle. Investigators later found that the failure

wasn't caused by a single bad actor. There was no villain, no individual to point the finger at and bring to justice.

One contractor installed the panels. Another supplied the bolts. A third tested them. A fourth signed off on the work. Each complied with their individual responsibilities. But no one verified that the system, as a whole, was safe.

This is a perfect illustration of what can happen with connected transactions.

Even when individual transactions are valid, and individual payments are flawless, the system can still fail if the dependencies between steps aren't visible or owned. The Big Dig didn't collapse because of a single error. It revealed what happens when responsibility stops at the edge of scope and no one is accountable for how those scopes interact.

In Chapter Three, we talked about hierarchy: transactions create meaning, flows create order, payments create movement. In Chapter Four, we discussed classification: different transactions behave differently and must be treated accordingly.

The Big Dig shows us what comes after a transaction is defined and classified, when the real challenge becomes coordination over time.

A dependency exists anytime one action assumes another has already happened, will happen next, or has been verified elsewhere. Most of the time, those assumptions feel reasonable. They're built on precedent, trust, and professional experience. Thoughts like, *Escrow releases will sync across time zones*. Or, *Counterparties will confirm receipt instantly*. But these assumptions blur under real-world conditions: different jurisdictions, different cutoff times, different currencies. What seems simultaneous on a deal calendar rarely is.

Over long timelines, small misalignments accumulate until the system is under stress.

When no one owns the sequence, everyone thinks someone else is checking it. Each participant performs their role, documents compliance, and moves forward. Progress continues. Momentum builds. But just like the Big Dig, the overall integrity of the system can quietly erode in the background.

In my experience, problems within dependencies hardly ever come because people lack skill or care. Instead, it's often because coordination is treated as implicit rather than designed.

The lesson here isn't that complex transactions should be avoided. After all, the Big Dig did finally succeed in creating a highway that buzzes underneath the city of Boston. But its weak dependencies led to over sixteen years of construction, and a final price tag of $14.6 billion.

The point is this: Complexity requires explicit ownership of dependencies. Someone has to be responsible for the order of operations, the timing of confirmations, and the conditions that make it safe for the next step to proceed.

That responsibility doesn't emerge naturally. It has to be designed into the transaction itself.

Defining Connected Transactions

At its core, a connected transaction is not a single movement of money. It's a set of actions that must coordinate across time, sequence, or condition in order to be safe. A dependency exists whenever one step cannot safely proceed without confirmation that another step has occurred, occurred correctly, or occurred in the right order. That confirmation might be about timing, evidence, authority, or receipt. But the common feature is this: something else has to be true before this step should move forward.

For example: A private equity exit doesn't happen as one wire transfer. Buyer funds, debt refinancing, escrow funding, and advisory fees all orbit a single close date. Each depends on the others to move in lockstep.

Most organizations already understand this intuitively. You can hear it in how people talk on closing calls. *"We're waiting on confirmation." "That release depends on receipt." "We can't move until legal signs off."* The awareness is there. What's often missing is a system that makes those dependencies explicit and visible.

In general terms, connected transactions follow two basic structural patterns:

First, we have **synchronous** groups. In this case, multiple actions must occur together, or not at all. Closings are the most familiar example. Funds, documents, and approvals are meant to move as a single event. If one piece fails to arrive, none of them should. If those transactions feel fragile to you, that's because they are. They rely on precise coordination across many parties at once.

Second, we have **sequential** transactions. One step unlocks the next. Construction financing is a good example of this. Funds are released after inspections. Inspections follow completed work. Completed work depends on prior funding. Each step creates the conditions for the next, and moving out of order introduces risk. Here, timing and evidence matter more than having everything happen at the same time.

Here's the catch: most real-world transactions aren't purely synchronous or sequential. They're **hybrids**.

At the top level, a transaction may look grouped: a bond issuance, an acquisition closing, a major disbursement. But underneath, it behaves like a flow. Regulatory approvals arrive at different times. Confirmations post asynchronously. Funds become available on different schedules depending on institution, jurisdiction, or rail.

The danger lies in how we treat these transactions. Many times, finance teams treat these as if all the pieces move together. But the process underneath it all is staggered.

What makes this especially challenging in modern finance is fragmentation. Each participant operates inside their own systems, calendars, and definitions of completion. One party considers funds "sent." Another considers them "received." A third considers them "available." Each is correct within their own context. The dependency lives in the mismatch.

Without an explicit dependency model, coordination depends on experience and judgment. That can work at low volume or with stable teams. But as complexity increases—more parties, more time zones, more automation—intuition becomes unreliable. The system moves faster than shared understanding.

You don't reduce that risk by adding more approvals at the payment level. You reduce it by making dependencies visible before execution begins—by deciding who waits on whom, what evidence matters, and what conditions must be met before the next step can safely proceed.

Once dependencies are explicit, coordination stops being a matter of hope. It becomes a matter of design.

Building Coordination Into Your Connected Transactions

The good news is that this doesn't require heroic effort or complicated tooling. Most of the fixes are simple. They just need to be applied deliberately, before money starts moving.

The first step is **mapping out your dependencies**.

This isn't a technical exercise. It's about being honest about which transactions are allowed to proceed on their own, and which ones are not. If one transaction assumes another has already occurred—funds verified, authority granted, terms locked—that dependency should be explicit, not implied.

A useful way to surface these dependencies is to walk through what happens when a prerequisite transaction is delayed or incomplete. What still executes? What should pause

automatically? If a downstream transaction can move forward even though its inputs aren't settled, that dependency probably hasn't been properly defined.

Next, it helps to **create a simple funds map**.

A funds map traces each dollar from its source to its destination, including any intermediate accounts, with expected timing along the way. Not just "sent" and "received," but when funds are expected to post, clear, and become usable. In complex transactions, timing matters more than people realize. Two parties can both be right and still out of sync.

Another quiet source of trouble is ambiguity around contingencies. Which is why it's important to **agree on specific contingency rules**.

When something unexpected happens—and in real transactions, it always does—teams often improvise. Someone makes a judgment call. Someone else assumes it's fine. That's not a failure of discipline. It's a sign the contingency wasn't agreed to in advance.

Explicit contingency rules help here. What constitutes a pause? What requires escalation? What can proceed with a workaround, and what can't? Deciding these things ahead of time keeps pressure from forcing bad tradeoffs later.

Closely related to this are **gatekeeper events**.

A gatekeeper event is a moment where the sequence is intentionally checked before anything else moves. Think of it as a natural stopping point: *We don't proceed past this step until these specific conditions are met.* Naming those moments creates rhythm. It gives everyone permission to slow down briefly so the rest of the process can move forward with confidence.

All of these are great principles in theory.

But in reality, you need someone to watch over the sequence and make sure it runs smoothly.

Enter: **the transaction coordinator**.

This role exists in many transactions already, even if it isn't formally named. It might be a closing attorney, a lead banker, a treasury director, or a project finance lead. What matters isn't their title. It's their authority.

The coordinator's job is not to move money. It's to protect the sequence.

They're the one who knows what depends on what, and who has the ability to say, *"We're pausing here until this is clear."* That pause isn't a failure. It's the system working.

This is a delicate but profound mindset shift that I've seen make a huge difference for finance teams.

The interesting thing about these steps is that, when they're in place, transactions don't feel heavier. If anything, I'd say they feel lighter.

Coordination doesn't add friction. It removes it.

Why Healthy Flows Should Look Boring

I don't know about you, but I didn't get into finance for fast-paced, adrenaline-pumping moments where your stomach drops and your life flashes before your eyes. If that was the goal, I should've become a bullfighter. Or a trapeze artist.

When transactions work well, there's no drama. No last-minute scrambling. No heroic save at the wire. Things move when they're supposed to move, in the order they're supposed to move, and everyone goes home on time.

That kind of execution often gets described with a word people don't typically associate with success:

Boring.

That is not an accident.

A good example is the 2018 sale of Toshiba's memory-chip business to a Bain Capital–led consortium. At roughly $18 billion, it was one of the largest technology transactions ever completed at the time. It involved multiple buyers, sellers, regulators, currencies, escrow accounts, and banks operating across continents and time zones.

On the surface, it looked straightforward. On the inside, it was anything but.

Dozens of disbursements had to occur in coordination. Regulatory approvals in Japan and the United States had to land in a narrow window. Escrow releases were conditional, not automatic. Nothing moved unless everything that needed to be true actually was true.

The team didn't rely on hope or timing luck. They rehearsed the sequence. They defined gatekeeper events. They configured conditional releases so funds simply couldn't move out of order. Everyone knew what they were waiting on, and who was responsible for saying "go."

When closing day arrived, the execution looked almost effortless. That wasn't because the transaction was simple. It was because the dependencies had already been handled.

That's what good coordination buys you.

When connected transactions are designed well, a few things start to change. Teams spend less time firefighting and more time doing real work. "Almost disasters" turn into non-events. Pressure doesn't disappear, but it becomes manageable. People stop relying on personalities and start relying on structure.

Most importantly, confidence improves.

Not the brittle kind that comes from optimism or familiarity, but the steady kind that comes from knowing the sequence is protected. When something unexpected happens—as it always does—there's a plan for how to pause, confirm, and proceed. No one has to improvise under deadline.

This is where trust really comes from. Not from assuming everyone will do the right thing, but from designing transactions so the right thing is the easiest thing to do.

Connected transactions don't require heroics. They don't need louder voices, faster clicks, or longer nights. They need someone to own the space between steps—and a system that supports that ownership.

Here's what I want you to take away from this chapter: A transaction is only as strong as its weakest dependency, and strength comes from making those dependencies visible, owned, and protected before money moves.

When that happens, execution gets calmer. Outcomes get better. And transactions—no matter how complex—start to feel a lot less stressful than they used to.

In the next chapter, we'll look at how authority fits into this picture, and why clearly defined permissions are what allow coordination to work without slowing everything down.

CHAPTER 6: Transaction Stages

Security isn't a moment—it's a sequence

A mid-market company was in the middle of acquiring a smaller competitor. Nothing out of the ordinary, just a solid strategic move. The kind of transaction teams handle all the time.

For weeks, the deal lived where most transactions live early on: spreadsheets, shared folders, and long email threads. Valuation models were evolving. Integration assumptions were being refined. Everyone involved understood that the numbers were still moving.

They were drafts.

At some point, the CFO forwarded one of these models to outside counsel with a short note: “Here’s where we’re landing—still iterating.”

It wasn’t a formal handoff. No one declared the transaction “live.” There was no moment where the team collectively said, this is now real. Work just continued, as it usually does.

Counsel began drafting purchase agreements based on the numbers they received. The bank used the same figures to prepare funding mechanics. Internal finance started setting up accounts so closings wouldn’t be delayed.

No one was acting irresponsibly. Each group assumed the others knew what they were doing. Momentum took over.

Two days before close, the CEO asked a simple question: “Why does the cash requirement look higher than what we agreed?”

Everyone had an answer. None of them matched.

The issue wasn’t a bad model or a careless mistake. It was something quieter. The transaction never crossed a clear boundary between *thinking* and *execution*. Draft materials had slipped into live use without ever becoming authoritative.

In people’s heads, the deal was still being shaped. In the system, it was already behaving like it was ready to close.

This scenario is hypothetical, but the pattern isn’t. It repeats in municipal bond deals, real estate closings, and corporate acquisitions. The team slowed down, reconciled documents,

and closed the deal a day later than expected. No one lost money, no one made the headlines.

But a fundamental truth of transactions became clearer: Teams don't lose control at the end of a deal. They often lose it earlier, when a transaction stops being treated as a sequence and starts being treated as a blur.

That's why stages matter.

Two Lifecycles Running at Once

Every transaction has two lifecycles running at the same time. The first is human. The second is operational. Most breakdowns happen when the two drift out of alignment.

The human lifecycle reflects how people actually make decisions. Whether you're financing a school, acquiring a company, or closing a real estate deal, the mental process follows a consistent arc.

Intent comes first—someone wants an outcome, options are being explored, nothing is binding yet. Then commitment, where parties are chosen and the decision becomes real. Execution follows, where documents are finalized and money is prepared to move. And finally settlement, when value changes hands and the obligation is discharged.

This is intuitive. It works. Except when the system around it doesn't reflect that process.

The operational lifecycle is where clarity either holds or breaks down. Most transaction systems fall short in the same way: they allow work from one phase to quietly bleed into the next. Draft thinking gets treated like execution. Assumptions start behaving like facts. By the time anyone notices, the transaction is already under pressure.

A well-designed transaction system acts as a control plane—the place where truth is anchored and decisions are carried out safely. That requires two distinct frameworks working together: one for the transaction as a whole, and one for the flow of funds within it.

Defining Transaction States: Open or Closed

At the transaction level, the model is deliberately binary: a transaction is either Open or Closed. This simplicity is intentional. It eliminates ambiguity at the governance layer. There's no "mostly finalized" or "essentially locked." Either the transaction is still subject to change, or it isn't.

An Open transaction is active and in progress. Participants are engaged. Information can still be updated. Verification is ongoing. Within an Open transaction, the funds inside it are

moving through their own structured lifecycle. But at the transaction level, the system still allows authorized changes. Authority exists to modify, extend, or restructure what's happening.

Open is where most of the work happens. It's also where most of the risk lives, because the system is still accepting input from the world.

A Closed transaction is complete. The record is locked. No edits, overrides, or adjustments are permitted—regardless of title or urgency. Once closed, the full history of the transaction—every change, approval, confirmation, and timestamp—becomes an immutable record for audit, defense, and learning.

The security function of Closed is straightforward: it removes discretion at the moment of highest institutional risk. If something wasn't right before the transaction closed, the answer is to resolve it while it's still Open—not to push through and clean it up later. That boundary does more to prevent abuse, error, and regret than almost any other control you can design.

The Four Stages of the Funds Lifecycle

Within an Open transaction, each flow of funds moves through its own lifecycle. This is where the operational detail lives—and where most of the coordination, verification, and execution work actually happens.

The four stages are Draft, Active, Ready to Fund, and Funded.

Stage 1: Draft

Draft is where thinking happens safely. It's the sandbox—the place where amounts are debated, structures are tested, and assumptions are challenged before anything is real.

Draft is intentionally forgiving. It's internal, exploratory, and iterative. You're allowed to change your mind here. In fact, I'd even say you're *supposed to*. At the same time, Draft material is not executable or verifiable. And it's definitely not a signal that anyone should act.

Many times, draft material escapes too early. I've seen this happen dozens of times, and you probably have too. A working spreadsheet gets forwarded. A preliminary number gets reused. Suddenly, something that was meant to support thinking starts driving action.

The security function of Draft is simple: keep half-formed ideas from becoming authoritative by accident.

The moment a flow of funds is shared with outside participants—couterparties, banks, counsel, vendors—it needs to leave Draft and move to Active.

Stage 2: Active

Active is where the flow of funds becomes real in the operational sense. Information is no longer just helping someone think: it's informing what people do next.

That change matters.

Once a flow is Active, assumptions aren't good enough anymore. Information has to be consistent. Roles have to be clear. And verification becomes mandatory—not as a task to check off, but as a condition of the stage.

This is the time to verify identity, validate accounts, and make sure approvals align with what's actually being executed.

Think about a real estate closing. Before funds move, someone confirms identities, wiring instructions, and authorizations. That doesn't happen because everyone is suspicious of one another. It happens because everyone understand the cost of being wrong.

That's what Active is for. It forces alignment before money moves. It makes manipulation harder because there's a shared, authoritative view of what's happening. And it prevents social engineering from sneaking in through informal channels.

If Draft is about thinking freely, Active is about proving readiness.

Stage 3: Ready to Fund

Ready to Fund is the stage that most transaction systems skip—and it's the one that matters most under pressure.

This stage exists to create a deliberate pause between "verification is complete" and "the wire goes out." It's a confirmed state, not just a status. Every required condition has been met. Every required authorization has been obtained. What's left is execution, not judgment.

This separation matters because the moment before funds move is also the moment of maximum pressure. Deadlines are looming. External parties are waiting. People want to be done. That urgency is exactly when controls get bypassed—when someone says "we've checked everything, just send it."

Ready to Fund removes that ambiguity. It says: we have checked everything, and here is the documented confirmation that we have. The decision to fund is no longer discretionary. It's the result of a process that already happened.

This stage also creates a clear audit trail for exactly the right moment. If a wire is ever questioned, Ready to Fund is where you look first. Who authorized it? When? Against what verified information?

When this stage is explicit, executing a transaction feels almost procedural. When it's missing, execution feels like a judgment call made under pressure. That's when mistakes happen—and when fraud finds its opening.

Stage 4: Funded

Funded is the simplest stage in description, and the most unforgiving in practice. This is where value moves. The flow of funds is complete.

Once a flow reaches Funded, nothing can change. No edits, no overrides, no "just fixing something real quick." If something wasn't right before this stage, the answer was to stop it at Ready to Fund.

The Funded stage also triggers the permanent record. The full history of this flow—Draft through Funded—becomes part of the closed transaction record. Everything is preserved: amounts, timing, approvals, verifications, and any exceptions that were noted along the way.

At its best, reaching Funded should feel almost boring. When execution feels dramatic, it usually means earlier stages didn't do their job.

How the Two Tracks Work Together

The transaction state and the funds lifecycle don't operate independently—they're designed to reinforce each other.

A transaction opens, and within it, one or more flows of funds begin moving through their own stages. A municipal bond closing might have a single disbursement. A complex acquisition might have a dozen, each moving at its own pace, each requiring its own verification. The transaction stays Open until all flows have reached Funded and all obligations are discharged.

When the last condition is met, the transaction closes—not as another step in a sequence, but as a permanent state change. The record locks. The archive becomes the source of truth.

This two-track structure also clarifies authority. While a transaction is Open, authorized participants can act within their role and stage. No one can skip Ready to Fund because a deadline is tight. No one can bypass verification because they trust the counterparty. The stage defines what's permitted—not the title of the person making the request. When the transaction closes, authority ends. The record speaks for itself.

How Stages Work in Practice

Stages only matter if people use them. And people only use them if they make work easier, not harder.

The mistake organizations often make is treating stages like policy—something written down, explained once, and then bypassed the moment pressure shows up. That's not what stages are for. Stages are a *playbook*. They exist to remove ambiguity at the moments when judgment is most likely to fail.

A good playbook answers three questions clearly:

- What should be happening right now?
- What shouldn't be happening yet?
- What has to be true before we move on?

Stages do exactly that.

When a flow is in Draft, change is normal and expected. When it's Active, verification is required. When it's Ready to Fund, execution is authorized. When it reaches Funded, the work is done.

That clarity reduces decision fatigue. People don't have to guess how careful they need to be or whether it's okay to make changes. The stage answers that for them.

Just as important, stages create shared expectations across teams. Legal, finance, operations, and external partners don't need constant alignment meetings if they all understand what each stage means. The structure does that work quietly in the background.

It's worth saying explicitly: none of this requires a specific platform.

With good software, stages tend to be reinforced naturally. Drafts stay in Draft. Active is explicit. Ready to Fund is a confirmed state. Funded is final.

Without software, the responsibility shifts to process and culture—harder, but entirely possible. What matters is not *how* stages are enforced. It's that they *are* enforced.

The Quiet Benefits of Stage Clarity

When stage discipline exists, most of the benefits don't show up as dramatic wins. They show up as the absence of problems that used to feel normal.

There's less panic at the finish line. When stages are respected, funding doesn't feel like a cliff. The hard thinking already happened earlier, where it belonged. That reduces the need for exceptions, escalations, and late-night approvals that no one wants to own afterward.

You also see fewer "who approved this?" moments, because approvals are tied to clear points in the sequence. Decisions are made in context, not reconstructed after the fact.

People know what's expected of them. They don't have to guess how careful they need to be or whether now is the moment to push back. The stage answers that question. That alone removes a surprising amount of stress.

There's also less reliance on memory and trust as controls. Systems carry the burden instead of people carrying it in their heads.

From a security standpoint, fraud attempts surface earlier, when they're easier to stop. Conflicts don't get resolved through debate or urgency—they block progression until they're resolved properly. Attacks fail because the sequence doesn't give them anywhere to hide.

That's the real win of stage discipline. It doesn't ask people to be more vigilant. It asks the system to be more honest.

In the previous three chapters, we've discussed how to improve clarity around Decisions in three specific areas:

- Transaction types
- Connected transactions
- Transaction stages

Of course, all of these decisions don't just execute themselves. So, the question that naturally follows is:

Who has the authority and responsibility to make decisions inside a transaction?

In the next three chapters, we're going to discuss the second aspect that affects the health and security of transactions: Governance.

Governance

This section is about aligning authority with how transactions actually move. You'll learn how to clarify internal responsibility, bring external relationships into the same structure, and define transaction-level roles so accountability doesn't drift under pressure. When governance is clear, transactions stop relying on familiarity and start relying on design.

In this section of the framework, you'll learn how to get clarity on:

- Organizations
- Contacts
- Participants

CHAPTER 7: Organizations

How internal authority is structured and enforced

In 2015, Ubiquiti Networks disclosed that it had lost $46.7 million to fraud.

At first glance, the story looked familiar. An employee received instructions that appeared to come from senior leadership. The request was urgent. The language was right. The timing made sense. Funds were moved.

What made the case unsettling wasn't what went wrong. It was what *didn't*.

The employee didn't bypass controls. They didn't break policy. They didn't act recklessly. They followed the rules exactly as written.

From a technical standpoint, everything worked. The employee had system access. They had approval authority. The transaction cleared. The software did what it was designed to do.

But from an organizational standpoint, something was off.

The employee never should have had the organizational authority to execute a transaction of that size or type on their own. Not because they weren't capable. Not because they weren't trusted. But because the transaction itself didn't belong to them—or to anyone else, clearly, end to end.

There was no single owner responsible for the transaction from intent through execution. No defined structure that reflected how the business actually made decisions. Authority converged in one moment, in one inbox, without a system around it.

This was a governance vacuum at the transaction level.

The most important detail in the Ubiquiti case is often missed: the employee followed policy. The policy simply ignored how the organization was supposed to work.

That's the pattern we're here to fix.

Why Governance Disappears When it Matters Most

We all want to believe that our organization is well-governed. Organizations have policies, approvals, audits, committees, controls.

And yet, when the moment comes for value to move—when money leaves an account, commitments become irreversible, and risk becomes real—governance quietly disappears.

Instead, you have emails, spreadsheets, phone calls, and urgency. All wrapped up in the best of intentions.

Here's the problem: most governance systems were never designed for execution.

Modern governance is largely retrospective. It's excellent at explaining what should have happened *after* something goes wrong. Audits reconstruct the past. Policies document intent. Committees assign responsibility once the outcome is already known.

What governance rarely does is constrain and coordinate behavior *before* value moves.

That gap explains why organizations keep repeating the same mistakes—even after frauds, audits, and new policies. When governance lives after execution, execution is left to habit. And habit, under pressure, doesn't change just because a new policy exists.

The mindset shift is how we should view transactions themselves.

Not as moments or tasks. But as **systems**.

A transaction is a temporary system formed around intent, trust, authority, and risk. And like any system, it only works when its core elements are defined *in advance*.

Before an organization can govern transactions, it must first adopt a transaction mindset—one that clearly defines:

- **Participants**: Who is involved, and in what capacity
- **Permissions**: What each participant is allowed to do
- **Contacts**: Who is allowed to exist inside the transaction at all
- **Stages**: How intent turns into irreversible reality

When these are defined, governance turns into something you can actually use.

Within this system, trust needs to be handled carefully. Trust is valuable: it allows teams to move quickly and collaborate well.

But trust is a belief. Structure is a guarantee.

The most resilient systems we rely on every day—aviation, accounting, engineering—don't depend on trust alone. They depend on defined roles, limited permissions, and enforced sequences. Transactions deserve the same treatment.

With that mindset in place, we can finally talk about organization—not as an org chart exercise, but as a way to give transactions a real home inside the business.

How Organizations Give Transactions the Home They Need

Authority does not flow through trust, competence, or seniority. It flows through organizational structure.

That may sound obvious, but it's easy to lose sight of in modern finance. When something needs to get done quickly, we tend to rely on the people we trust most, the ones who are capable, responsive, and experienced. Over time, those people accumulate more access, more responsibility, and more informal authority—often without the structure changing around them.

That's how gaps form.

Before an organization can be organized, it must first learn how to think in transactions. Most organizations try to solve transaction problems by rearranging departments, adding controls, or layering approvals. But organization without transaction thinking is like rearranging musicians on a stage before anyone agrees on the song.

The result is rarely harmony. It's usually louder confusion.

Transaction thinking has to come first. Only then can organization and governance serve their proper role: enabling execution, not distorting it.

One of the most common reasons transactions fail is surprisingly simple: they don't have a home.

They live between departments. They pass through inboxes. They're tracked across spreadsheets, email threads, accounting systems, and people's memories. Everyone touches the transaction, but no one clearly owns it.

When transactions don't have a home, a few predictable things happen. Overspending can creep in because no one is watching the whole. Proceeds can get misapplied because responsibility is fragmented. One department may begin to dominate execution, not because it's malicious, but because it's closest to the moment when action is required.

This is, at its core, a design problem.

True organizational governance starts by giving transactions a place to live—by deciding where ownership sits, how responsibility flows, and how authority is distributed *before* execution begins.

This is where organizational structure actually matters—but only in the right order.

Organizational units are containers. These include:

- The organization
- The team or division
- The individual member

They exist to support transactions.

Transactions are the thing being contained. Organizational structure provides the boundaries within which transactions can be executed safely and repeatedly.

When those boundaries are clear, authority makes sense. A team knows what kinds of transactions it can own. An individual knows which stages they're responsible for. Escalation paths are obvious, not political. Saying "this isn't my authority" feels normal, not obstructive.

When those boundaries are unclear, the opposite happens. Authority bleeds across teams. Individuals inherit responsibility by proximity. Systems reflect convenience rather than business reality. That's exactly how technically authorized but organizationally misaligned actions slip through.

Think of an orchestra warming up. You hear sound. You see activity. Everyone is clearly skilled. But nothing quite comes together.

Transaction thinking is the sheet music. It defines what's being played, when different sections come in, and how their parts fit together. Once that structure exists, organizational roles stop competing for control and start working in sequence.

But even with sheet music, something is missing.

If one section takes the lead—especially the loudest one—the performance becomes distorted. Timing slips. The music loses balance.

That's when you realize the final requirement: someone has to be accountable not to a department, but to the transaction itself. Someone has to guide the orchestra so that everyone follows the music.

That's the role of the conductor.

The Missing Role: The Transaction Coordinator

In an orchestra, the conductor doesn't play an instrument. They aren't the loudest voice in the room. They don't belong to the strings, the brass, or the percussion.

Their loyalty is to the music itself.

In organizations, failures can happen because no one is accountable to the transaction as a whole.

Each function sees its piece. Finance looks at the ledger. Legal looks at the documents. Treasury looks at cash movement. Operations looks at execution. All of that work matters.

All of that is rational. All of it is well-intentioned. And all of it can still produce distortion.

The transaction itself—the thing tying intent, authority, and value together—often belongs to no one. What's missing is a role that optimizes for execution.

In an orchestra, a conductor's job is simple to describe, even if it's hard to do well. They keep time. They manage sequence. They ensure that each section enters at the right moment and steps back when its part is done.

In transaction terms, this means one role is accountable for:

- The integrity of the transaction from intent through settlement
- Knowing what stage the transaction is in
- Knowing who is allowed to act at that stage
- Ensuring authority doesn't drift under pressure

This person becomes the transaction coordinator we mentioned in Chapter 5.

Without a transaction coordinator, organizations rely on informal coordination. People step in because they're capable. Others defer because they're busy. Decisions get made by whoever is closest to the inbox when urgency peaks. Over time, that becomes the system—even though no one would design it that way intentionally.

Having a clear role makes that coordination explicit and repeatable. And when they do their job right, governance feels less like policing and more like keeping time.

Most organizations assume this role exists implicitly. They believe ownership will emerge naturally. That someone senior will step in. That good people will coordinate when it matters.

Sometimes that works. Often it doesn't—especially as organizations scale, transactions grow more complex, and execution windows compress.

When no one is explicitly accountable to the transaction itself, governance defaults to hierarchy, habit, or urgency. That's when technically authorized actions drift away from organizational intent, even when everyone is acting in good faith.

The transaction coordinator closes that gap. If you don't have this role explicitly, it's worth the time and effort to train people on your team to do the work of this role.

It creates a single point of responsibility without creating a single point of failure. It gives transactions a steward whose job is not to approve everything, but to ensure the system around execution is doing what it's supposed to do.

When that role is in place, the rest of organizational governance finally has something solid to attach to.

Aligning Organizations for Transaction Governance

Once the transaction has an owner, the next step is to align the organization around that role. This is where governance becomes practical.

The goal here isn't to redesign your entire company. It's to make sure authority, capability, and execution are pointing in the same direction when transactions move.

The simplest way to do that is to think in layers.

Every well-governed organization aligns authority across three layers: **Business Roles**, **System Permissions**, and **Approval Workflows**. When these layers match, execution feels natural.

First, business roles define real authority. Not titles, but responsibility.

What types of decisions does this role legitimately make in the normal course of business? What kinds of transactions should it own? What impact does it have on the organization if something goes wrong?

Governance begins in clarity. If a role can't clearly explain what it is responsible for in a transaction, it shouldn't have authority at that stage.

Second, system permissions translate business authority into technical capability.

This answers questions like:

- Which accounts can be accessed?
- Which transaction types can be initiated?
- Which dollar limits apply?

This is where many organizations drift. They start to accumulate permissions because it's convenient, and temporary access never gets revoked. Over time, systems stop reflecting how the organization actually works.

The gap between business authority and technical capability is where failures emerge. When someone can do more than their role should allow, the system quietly invites mistakes—especially under pressure. That's exactly what happened to Ubiquiti in 2015: the systems worked, but the structure didn't.

Third, approval workflows connect individual permissions into an organizational process.

They enforce sequencing. They require collaboration at the right moments. They ensure that no single person can carry a transaction beyond their context or expertise.

Good workflows don't slow execution down. They prevent momentum from outrunning understanding.

When these three layers align—business roles, system permissions, and approval workflows—authority becomes visible. Responsibility becomes shared. Execution becomes repeatable.

The good news is that this alignment doesn't require a massive transformation. All it takes is a few intentional mindset shifts.

Here are specific steps to help you get started:

- **Treat transactions as first-class objects.** Name them. Define their sages. Decide who owns them from intent through settlement. If a transaction matters, it deserves structure.
- **Assign a trained transaction coordinator.** This role should be accountable to the transaction itself, not a department. Their job is coordination, sequencing, and integrity.
- **Map roles by stage, not by title.** Ask who is responsible at initiation, verification, approval, and release. The same person shouldn't automatically own every stage because it's convenient.
- **Align permissions to authority.** If a role doesn't have business authority for a type of transaction, it shouldn't have the technical ability to execute it. This isn't about distrust. It's about fit.

- **Review permissions as roles evolve.** Organizations grow. People get promoted. Responsibilities change. Permissions should change with them. Regular reviews prevent yesterday's structure from becoming today's risk.
- **Prevent departmental dominance.** No single function should control a transaction end to end. High-impact transactions should naturally require collaboration across roles that bring different context and expertise.
- **Design governance into systems, not supervision.** Use digital identity, roles, permissions, and workflows so correct execution is enforced by the system itself, not dependent on managers catching mistakes after the fact.

When organizations take these steps, governance feels more supportive of the team's day-to-day work.

How Work Changes When This is Done Right

None of us enjoy the feeling of wondering whether an aspect of a transaction was our responsibility, or whether someone else has it covered.

And no one wants the load of a transaction landing on their shoulders when it's not part of their role.

When organizations are aligned in the model we discussed above, transactions gain clarity before urgency shows up. People know who owns what. They know which stage they're in. They know when it's their turn to act—and when it isn't.

Questions that used to feel political become procedural. Departments stop competing for control. Finance doesn't have to overreach to feel safe. Legal doesn't have to slow everything down to maintain rigor. Treasury doesn't have to improvise under pressure. Operations doesn't have to push past structure just to keep things moving. Each group contributes its expertise at the right moment, then steps back.

Most importantly, people feel protected by the system.

They're no longer forced to rely on instinct, trust, or seniority cues when something feels off. The structure gives them cover. It gives them a reason to pause. It gives them a clear path to escalate without fear of being seen as difficult or slow.

Risk doesn't live in departments, titles, or policies sitting on a shelf.

It lives in transactions—the moments where value moves and decisions become irreversible.

When organizations treat transactions as first-class systems, give them owners, and align structure around execution, governance stops being theoretical. It becomes something you can feel in the flow of work.

That's what this chapter is really about: giving transactions a home inside the organization, so authority flows through structure instead of trust, and execution reflects how the business actually works.

Once that internal foundation is in place, the next question becomes unavoidable.

Who is allowed to be part of the transaction at all?

CHAPTER 8: Contacts

Who is outside your organization, and how are they recognized?

Organizations don't just wake up one day and decide to lose their way.

What usually happens is a quiet drift.

A company brings on outside capital to speed up their growth. The investors are smart, the terms are fair. So, new board members join. Then come advisors and consultants. A few early customers help shape the roadmap. Vendors appear who "know how to scale."

None of this feels dangerous in the moment. Actually, it mostly feels helpful and exciting.

But over time, the center of gravity starts to shift.

Transactions that were once evaluated based on operation fit or long-term value start getting filtered through different questions. Will this help the next round? Does this support the valuation story? Can this move faster, even if it bends how the business actually works?

Each choice makes sense in isolation. And no one is acting in bad faith.

But, transaction by transaction, things start to shift.

External relationships begin to shape which deals are prioritized, how risk is tolerated, and whose timeline matters most. The organization becomes oriented around forces it never explicitly chose to serve.

Longevity isn't determined only by what happens inside your walls. It's shaped by how you organize and govern the people and businesses you invite into your transactions. External relationships carry weight. And when that weight isn't acknowledged or structured, it quietly pulls the organization off course.

The Real Weight of Contacts

It's easy to think of contacts as a simple inventory of people and companies you interact with. Vendors. Advisors. Board members. Customers. Investors. Names in a database, numbers on a sheet.

But in reality, that's not exactly accurate.

Every contact is a source of gravity.

Every external relationship pulls on your organization in some way. Who sits on your board influences how decisions get framed. Who invests in your business affects which risks feel acceptable. Who you listen to for advice shapes which paths seem reasonable. Even your earliest customers and vendors quietly influence what kind of organization you become.

In many cases, this is positive. These gravitational forces extend an organization's life. They strengthen discipline, sharpen focus, and reinforce purpose.

In other cases, they may keep the organization roughly where it is—comfortable, stable, but not necessarily resilient.

And some, often without anyone intending it, slowly shorten the runway by pulling the organization away from what it was originally built to do.

Transactions don't happen in isolation. They sit at the intersection of people, incentives, authority, and trust. Contacts are where those forces converge. And that's why, in many organizations, the tail ends up wagging the dog.

When contacts lead transactions, execution becomes reactive. The organization adapts itself around relationships instead of letting the transaction define the need, the risk, and the structure.

Reverse that order, and your clarity improves almost immediately. The transaction defines what information matters, which controls are necessary, and which external parties actually belong in each deal. People align to the transaction instead of the transaction bending around people.

Here's an example of where I've seen this dynamic play out over and over again: external capital.

When outside investors hold most of the equity, organizations often orient around growth, valuation, and exit. That can be perfectly rational from an investor's perspective. But it can also introduce real tension with employees and customers. Advice gets subtly shaped by timelines that aren't the company's own, and the long-term mission starts to get fuzzy.

That doesn't make outside capital bad, per se. But it is a powerful gravitational force.

When an organization is anchored in its purpose—its customers, its employees, and the work it exists to do—transactions tend to serve the business itself. Advice improves. Incentives become easier to see. External contacts still influence decisions, but they do so within a clearer frame.

Contacts will always exert gravity. The question is whether that gravity is intentional and aligned with the transactions that actually matter, or whether it's allowed to accumulate quietly until the organization no longer recognizes how it got where it is.

That's the problem contact governance is meant to solve.

Why Contacts Are So Hard to Govern

If you look closely at how most organizations actually store and manage their external contacts, the picture is rarely clean.

Some contacts live in the ERP. A few are duplicated. Others are half-filled because no one ever came back to clean them up. Sales has its own version of the world in the CRM—relationships built over time, notes saved in free text, details only one or two people really understand. Payroll lives in a spreadsheet that only HR touches. Vendors get set up in accounts payable by someone who's never spoken to them directly.

None of this is unusual.

Relationships develop organically. People save information where it's most useful to them in the moment. Trust builds over time. Someone "knows the vendor," or "has worked with them before," or "has always handled that relationship." And so contact information spreads across systems, inboxes, and personal memory.

What's missing isn't effort or care. It's a shared system for contact governance—something everyone can look to and understand how external parties should be identified, validated, and used in transactions.

That absence shows up in three predictable ways.

First, **contacts fragment because organizations fragment.**

Different departments maintain different relationships for different reasons. Sales tracks leads and customers. Operations manages vendors. Legal works with outside counsel. Finance sees names and bank accounts. Individual employees rely on personal networks they've built over years.

Each view is valid. None of them is complete.

When a transaction comes together, it pulls pieces from all of these places at once. The relationship context lives in one system. The payment details live in another. The approval happens somewhere else. Identities blur. Assumptions fill in the gaps.

No one is acting irresponsibly. The structure simply doesn't support a unified view of who this external party is *in the context of this transaction*.

That can lead to misalignment.

Second, **the systems we use aren't built for transactional contact governance.**

The tools most organizations rely on weren't designed to solve this problem.

ERPs do a good job of connecting departments. CRMs help manage relationships. Accounting systems track money. Each system works within its own domain.

But transactions cut across all of them.

A transaction brings together a relationship, a purpose, a counterparty, an approval, and a movement of value. When systems aren't designed around that moment, contact data stays scattered and context gets reconstructed manually—often under time pressure.

That's when people fall back on what feels safest: precedent, familiarity, or trust.

Lastly, **banks won't be there to catch you when you slip up.**

It's also easy to assume the bank will catch anything that matters. Banks verify identities. They perform mandatory due diligence, like Know Your Customer (KYC) and Know Your Business (KYB). When they move money, they do it securely.

But when banks do KYC and KYB onboarding for new customers, they're only getting to know the sender. Unfortunately, they don't do this for *receivers*. That's a huge reason we keep seeing wire fraud: your bank doesn't know who you're sending money to.

Beyond that, banks can't see the business relationship behind the payment. They don't know why this transaction exists, how this counterparty fits into the organization's work, or whether this contact makes sense for *this* transaction.

They can't see purpose. And they're not meant to.

That responsibility belongs to the organization.

When contacts are scattered, systems are disconnected, and external parties lack context, transactions lose their anchor.

They move forward because the pieces line up, not because the purpose is clear. Contacts gain influence through proximity and familiarity. Decisions happen without a shared understanding of what the transaction is actually meant to accomplish.

Until contacts are organized around transactional purpose, governance will always feel incomplete. The structure needed to answer *why this contact belongs here* simply doesn't exist yet.

That's what organizing contacts around transactions is designed to restore

Organizing Contacts Around Purposeful Transactions

Every transaction has a reason.

Money doesn't move randomly. It moves because something is being built, acquired, repaired, delivered, resolved, or settled. That underlying intent—the *purpose* of the transaction—is what gives it shape. And without understanding that purpose, it's almost impossible to organize contacts in a way that makes sense.

When purpose is clear, it creates a boundary that adds clarity to the rest of the transaction.

There's a well-known study in child psychology about playgrounds that illustrates this point perfectly.

When a playground had a clear fence, children ran everywhere. They explored. They pushed boundaries. They used the full space. When the same playground had no fence, children stayed close to the center. Movement slowed. Exploration dropped.

The fence didn't restrict freedom. It created it.

Purpose works the same way in transactions.

Without clear transactional purpose, people hesitate. Contacts exert outsized influence. Decisions get shaped by whoever is closest, loudest, or most familiar. Movement slows because no one is quite sure how far they're allowed to go.

When purpose is clear, behavior changes. People move faster. They explore options. They make better decisions because the boundaries are visible. Contacts are invited in for a reason, not just because they're known.

If you invite contacts into your ideas and desires, your purpose will eventually be repurposed by theirs. If you gather contacts around a clear transactional purpose—even an imperfect one—you create alignment. You unlock contribution without surrendering control.

Many organizations get stuck because they approach this the wrong way: their transaction playground is missing its fence.

They try to organize contacts as static relationships instead of dynamic participants in a specific transaction. But contacts only make sense in context. And context starts with purpose.

When contacts are organized around that purpose, governance stops feeling restrictive and starts doing what it's meant to do: helping transactions succeed.

How to Turn Contact Chaos into Transactional Clarity

Once you accept that contacts shape transactions—and that transactions shape the future of the organization—the question becomes practical very quickly: *what do you actually do differently?*

The answer starts with a shift in how you think about systems.

In the last chapter, we talked about how organizations need a digital identity internally. That same logic applies to how you treat external contacts.

External organizations and people participate in transactions. And those transactions deserve the same level of structure, clarity, and intent as anything happening internally.

What changes is about creating a system that determines who is allowed to participate in a transaction—and why.

Contacts must be governed procedurally, not socially.

Here are five steps to do this:

Step 1: Treat external contacts as part of the transaction system. Every organization already has a transaction system—the question is whether it's designed or accidental. External organizations and people should be onboarded through a single, intentional contact system tied to transactions. Other tools like ERPs, CRMs, and payment systems should consume this information, not redefine it.

Step 2: Be explicit about entities and individuals. External parties always exist in two forms: entities you transact with, and individuals you interact with. Both matter, and both need to be visible. Trust in a person should never substitute for clarity about the entity, and familiarity with an entity should never automatically extend to every individual. Group them together intentionally—but never confuse one for the other.

Step 3: Approve contacts contextually, based on transaction relevance. Approved vendor lists assume permanence, universality, and context-free trust—and transactions don't work that way. Someone can be appropriate for one transaction and entirely wrong for another.

Instead of asking whether you know a contact, ask whether they belong in *this* transaction. Approval must be contextual, authorization must be transactional, and relationship familiarity should never replace relevance to purpose.

Step 4: Build process, not encyclopedic knowledge. You don't need people who remember everything—you need systems that make the right decision obvious. Onboarding external contacts should be procedural and repeatable, with information flowing automatically into downstream systems. This removes guesswork and reduces reliance on memory.

Step 5: Re-evaluate contacts as purpose changes. Transactions evolve, and so should the contacts around them. A new transaction brings a new purpose, and a new purpose brings a new risk profile. Contact approval should move with that context rather than lingering indefinitely. Approval should expire with purpose.

Align With the Contacts that Extend Your Organization's Lifetime

When we focus on purpose and then use that context to choose our contacts, we reintroduce the right hierarchy of authority within a transaction.

Instead of bending transactions around relationships, relationships align to what the transaction actually requires. Contacts no longer pull the organization off course through proximity or influence. They reinforce direction instead.

When the boundaries are clear, teams explore more freely inside them. They escalate less. They hesitate less. They don't have to rely on personal judgment to decide what's acceptable. The system gives them cover.

Most importantly, external relationships start extending the organization's life rather than distorting it.

With organizations aligned around transactions, and contacts governed by purpose, one question remains.

Who, exactly, is allowed to participate—and how?

CHAPTER 9: Participants

Who is involved in a transaction, and how?

In 1914, Ernest Shackleton set out on what was supposed to be one of the boldest expeditions of the modern era: the first land crossing of Antarctica.

The plan was ambitious. The environment was unforgiving. And by any definition, the risk was enormous.

Then the mission collapsed.

The *Endurance* never reached land. The ship became trapped in pack ice, was slowly crushed, and eventually sank. Shackleton and his crew were left stranded on the ice, thousands of miles from help, with no realistic rescue plan.

And yet—every single man survived.

This wasn't because Shackleton accepted more risk. It was because he **removed it**.

Once the ship was lost, the original objective no longer mattered. Crossing Antarctica was no longer the transaction. Survival was.

At that moment, Shackleton did something most organizations struggle to do under pressure: he **redesigned participation**.

Authority became explicit. Roles were reassigned. Individual initiative was constrained. No one acted just because they could. No one stepped outside their role, no matter how capable or experienced they were.

Unauthorized initiative disappeared. Participation wasn't a suggestion. It was governed. That discipline—not heroics, not optimism—is why the story ends the way it does.

Now, contrast that well-known success story with something closer to home: what happened to Toyota Boshoku Corporation.

This major parts supplier and subsidiary within the Toyota Group lost approximately $37 million to a business email compromise scam in 2019.

This wasn't a chaotic startup. In fact, Toyota is known for its operational rigor. And yet, the fraud worked.

The attackers impersonated a trusted business partner and sent revised payment instructions. The request didn't bypass controls. It moved through them.

Emails were sent. Approvals were granted. Funds were released.

From a system perspective, everything looked normal.

What failed wasn't technology or policy. It was **participation design**.

Initiation, approval, and execution collapsed into a flat list of "authorized users." Being *in the workflow* became equivalent to having authority. Visibility implied permission. Familiarity replaced verification.

No single person clearly owned the transaction end to end. No one was explicitly responsible for stopping it. Too many people could participate—and that participation was never intentional.

This is how modern transaction risk usually shows up: as a system that quietly allows too many people to act without clear authority, context, or constraint.

Shackleton's environment was objectively dangerous. Toyota's was not.

And yet, we tend to assume the opposite about risk.

We treat large, important transactions as inherently risky because they involve many people, many systems, and many handoffs. So we respond by adding approvals, adding tools, and adding process.

What we rarely do is stop and ask a simpler question: *Who is actually allowed to participate in this transaction—and how?*

Shackleton survived Antarctica by tightening participation as conditions worsened.

Most organizations do the opposite. As pressure rises, participation expands. More people are copied. More people get access. More people are allowed to act "just this once."

That's when risk accelerates.

Because participation is not the same as authority. And being "in the workflow" is itself a form of power.

Until participation is designed deliberately, the most important transactions will continue to feel dangerous, because they are open.

A Practical Way to Think About Participation

Every transaction has participants, whether you name them or not.

This is true regardless of your org chart or the maturity of your systems. The moment value is allowed to move, a participation model already exists. The only question is whether it was **designed** or **assumed**.

Unfortunately, most organizations assume participation.

They rely on titles, precedent, inbox proximity, or system access. Someone is “involved” because they’ve always been involved. Someone else approves because that’s what the workflow routes to. Another person executes because they happen to be closest to the system when the moment arrives.

In these cases, risk is easily introduced because people act outside their role without even realizing they’ve done so.

To make participation governable, you have to strip it down to what actually matters inside a transaction: how each individual is allowed to participate.

Participation isn’t the property of a person or a department. It’s the property of a specific transaction.

The same individual may participate very differently depending on the transaction’s purpose, size, timing, and risk. And that’s normal. What’s dangerous is pretending participation is static.

These roles aren’t invented for this framework—they map directly onto the flow of funds introduced in Chapter 3. Remember: a flow of funds describes how money is meant to move through a transaction. That includes the order of events, the conditions at each step, and what happens when something changes.

Sender, Clearer, and Receiver are the roles that bring that structure to life. They turn an abstract flow into a set of accountable positions. View-only access extends the structure to anyone who needs context without the ability to act. Together, these four roles are how a flow of funds gets populated:

- **Sender:** The party that initiates or executes the movement of value. This could be an individual, a team, or a system acting on behalf of the organization. The sender commits capital.
- **Clearer:** Not every transaction has a clearer. But when one exists, this role verifies, validates, sequences, or releases the transaction. A clearer may temporarily receive

funds before redistributing them, or may simply act as the gatekeeper that determines whether the transaction is allowed to progress.

- **Receiver:** The external endpoint. A vendor, counterparty, lender, employee, or beneficiary. The receiver is where value ultimately lands. This role is often treated as passive. It shouldn't be. The receiver defines half the transaction.
- **View-only (shared access):** This is the ability to see a transaction without the ability to act on it. This is a sharing function, not a participation role. A participant can and should be able to share a transaction with members of their own team. But sharing doesn't grant authority. A view-only recipient has context; a participant has a role. Keep these categories separate to prevent transactions from accumulating people who were meant to observe but end up able to act.

When a flow of funds is built, it isn't complete until these roles are assigned. The Sender defines where value originates and who commits it. The Clearer—when present—defines the verification and sequencing point in the middle. The Receiver defines where value lands and who receives it. Each payment inside the flow inherits this structure. THat's the connection between participation and execution: the role tells the system who is authorized to act at each point in the flow, not just within the organization, but across every party the transaction touches.

The most common failure pattern I see? When systems collapse all of these roles into a single concept: **authorized user**.

That means, if you can see the transaction, you can touch it. If you can touch it, you can move it. And if you can move it, you have authority.

That's how participation quietly turns into power.

Attackers don't need to break the system when the system already allows too much participation. They just need to step into a role that was never clearly defined. That's exactly what happened in the Toyota case. And it's what happens, in quieter ways, every day.

How do you fix this?

Simple: recognize that participation does not equal authority.

Being included in a workflow feels benign. Often it's meant to be helpful. But inclusion without constraint is not neutral.

Being "in the loop" allows:

- Timing influence
- Information asymmetry

- Soft pressure on decision-makers
- The ability to normalize unusual requests

None of that requires approval rights.

Participation itself shapes outcomes. That's why it must be intentional.

Once you see this clearly, you can start placing boundaries where they actually work.

How to Turn Participation into a Governance Practice

There's one key principle that makes the biggest difference here: Make participation explicit.

Most organizations have plenty of controls. But they're still failing at governance because participation is assumed. People are allowed to act because access exists, because precedent exists, or because no one ever stopped to say, *this is where your role ends*.

Participation becomes something you inherit rather than something you're assigned.

One directional mistake is worth naming here, because it's common: pulling contacts into a transaction and assuming that makes them participants. It doesn't. As we saw in Chapter Eight, contacts are the roster of external relationships your organization maintains—vendors, counterparties, advisors, banks. Participants, on the other hand, are specific people authorized to act inside a transaction.

The direction runs one way: a participant, once their role in a transaction is complete, may become a contact in your system for further use. But a contact doesn't become a participant simply because you know them, or they're available. Skipping that distinction is how transactions end up populated with people whose presence was assumed rather than assigned.

The moment you decide to make participation explicit, governance stops being abstract. It becomes something you can actually work with.

The first shift is in your mindset. We talked about this in Chapter Six: a transaction is more than the moment a money moves. It's the container for a sequence of events. And sequences only work when the moments that matter are separated on purpose.

Every meaningful transaction passes through three distinct points: Initiation, approval, and release.

Most systems blur these together. That's where risk concentrates.

Initiation is the moment a transaction is proposed or constructed. It's when someone defines *what* should happen—who is being paid, for how much, and why. This is a creative act. It requires context, understanding, and judgment. But initiation should never commit capital on its own.

Approval is about intent. It answers a narrower question: *Does this transaction make sense for the organization, given what we know right now?* Approval should be informed, but it should also be limited to the right role: ideally, the transaction coordinator. At this point, they authorize progression, not execution.

Release is the irreversible moment. It's when value actually moves. Once this happens, the organization is exposed. Recovery becomes expensive or impossible. Release should be mechanical, deliberate, and protected.

When one person—or one role—can initiate, approve, and release, the transaction becomes fragile. Of course, I'm not saying that person is untrustworthy (necessarily). The problem is surface area. One role spanning all three moments creates exposure that no amount of good intent can fully offset.

Most organizations already apply this logic internally without thinking about it. Accounts receivable and accounts payable are kept separate. The person who requests a purchase order isn't typically the one who approves it or cuts the check. Payroll and treasure are distinct functions.

These separates exist because the cost of collapsing them—through fraud or honest error—is well understood inside the business. But transactions, because they sit outside normal operations and cross departmental lines, tend to fall through that logic. The same discipline that protects accounting doesn't automatically extend to the closing table. The safeguard that feels obvious inside the organization gets forgotten the moment external parties are involved. That gap—between internal risk controls and transaction controls—is where a disproportionate share of institutional risk quietly lives.

Separating these moments doesn't mean adding bureaucracy. It means assigning **different permissions to different stages**, even when the same person may be involved in more than one under controlled conditions.

Initiating a transaction should not imply approval. Approving a transaction should not imply release. Releasing a transaction should not imply authority over its purpose.

Once those distinctions are clear, participation starts to snap into focus. You can see who is expected to act, and just as importantly, who is not.

That's when another quiet risk usually appears: people who *can* act but aren't *meant* to.

Every organization has them. Legacy access. Emergency permissions that never expired. People copied into workflows “just to be safe.” None of this feels dangerous day to day. But in a high-pressure moment, these silent participants become pathways the system never intended.

Making participation explicit means removing that ambiguity.

If someone is not expected to initiate, they shouldn’t be able to. If someone is not expected to approve, they shouldn’t be asked to. If someone is not expected to release, the system shouldn’t allow it.

In short, align expectation with capability.

Now, there’s a type of permission I’ve missed so far in this discussion, but it’s more important than most organizations realize: the ability to stop.

In many environments, stopping a transaction feels like insubordination. Like you’re the one causing a problem, or making a big deal out of nothing. Nobody wants to be in that position. So, people hesitate because they’re unsure whether they’re allowed to slow things down.

That hesitation is exactly what social engineering exploits.

Well-designed participation makes stopping legitimate. When someone senses an anomaly—an unusual request, a timing mismatch, a detail that doesn’t fit—the system should give them explicit permission (and the ability) to pause execution. Sensing a problem should be built into the role.

When stop authority is explicit, people use it calmly. When it’s implicit, they don’t use it at all.

This is what participation mapping really is: a governance decision about where authority begins, where it ends, and how value is allowed to move in between.

Once participation is intentional, transactions stop relying on instinct. They rely on structure.

And that’s when governance starts doing what it was always supposed to do: protect the work without getting in its way.

What Changes When Participation is Designed Correctly

Here’s a situation that has happened many times, in many forms.

A large organization (like a municipality, a school district, or a nonprofit) raises a good amount of capital for a specific set of long-term projects. The money comes with expectations—sometimes contractual, sometimes regulatory, sometimes simply moral. It's meant to be used for a clearly defined purpose.

At first, everything works fine. Projects begin. Payments flow. Reports reconcile. On paper, the system seems to be working.

But over time, small decisions start to drift.

Funds intended for one initiative are temporarily redirected to another. A restricted source is used where it technically shouldn't be, but it's justified internally as a short-term bridge. Someone tells themselves it will be corrected later.

Then pressure increases.

Deadlines tighten. Results are expected. The original assumptions no longer quite hold. To keep things moving, adjustments are made. Entries are reclassified. Transfers are layered. What began as a workaround slowly becomes a pattern.

At some point, new capital is raised or new commitments are made based on information that no longer reflects reality. Now, the organization has crossed a line it can't easily step back over.

Investigations follow. Leadership changes. People are removed. In some cases, consequences extend far beyond the individuals directly involved.

Here's the point: Good people can do deeply damaging things **simply because the system lets them**.

The system allowed people to move money in ways they shouldn't have been able to. It allowed accounting to conceal misuse instead of exposing it. It treated access as authority and familiarity as permission.

No one designed it that way—actually, no one designed it at all.

Of course, there's a more sinister version of this story. A trusted employee has broad capability. They can initiate transactions. They can approve them. They can release them. Over time, small amounts are diverted. Then larger ones. Adjustments are made to hide the trail.

Eventually, it's discovered. And once again, the post-mortem asks: "How could this happen without anyone noticing?"

The answer is always the same.

Because they were able to do it.

Here's the point: When the system removes the ability, it protects *everyone*—including the innocent.

When initiation, approval, and release are separated by design, no single role can redirect funds without visibility. Bad decisions don't compound because they can't be quietly hidden.

Of course, we know that permissions help protect teams from fraud, like what happened to Toyota Boshoku. But it also prevents good people from drifting into irreversible mistakes.

When participation is designed correctly, departments get to focus on what they're good at. Accounting stays focused on accounting. Finance stays focused on finance. Operations stays focused on building and delivering.

Transactions become the connective tissue—not the pressure point.

This is the upside of transaction thinking. It's about creating clear, humane boundaries that let people do meaningful work without putting themselves—or the organization—in impossible positions.

And that's when transactions stop being the thing that quietly brings organizations down—and start becoming the thing that lets them build, grow, and last.

Governance comes into focus when it is built around the moment value moves. When transactions have a clear home, when external relationships are organized by purpose, and when participation is intentionally designed, execution becomes calmer and more reliable. Authority is visible. Responsibility is shared. People know where their role begins and where it ends. The system supports judgment instead of depending on it, allowing individuals and teams to do their best work without carrying unnecessary risk.

In that environment, governance becomes something you feel in the flow of work—quiet, supportive, and durable.

With participation clearly defined, the next question is straightforward: how do you verify that the participants in a transaction are truly who they claim to be, and that money is going to the right account?

Integrity

This section is about making transactions dependable under real-world conditions. You'll learn how to anchor decisions in proof, keep information connected as it moves, choose execution paths that preserve options, and limit exposure when things don't go as planned. When integrity is built into the process, transactions become calmer, faster, and easier to trust.

In this section of the framework, you'll learn how to get clarity on:

- Verification
- Information
- Payment Rails
- Transaction Insurance

CHAPTER 10: Verification

Proof outlasts every impression

When something goes wrong, people usually jump straight to the mechanics. Which system failed? Which control didn't fire? Which approval should have caught it?

Those questions matter, but they're rarely the first place things actually broke.

More often, the problem shows up earlier—before amounts, before timing, before routing.

Who was actually on the other end?

In 2014, Scoular, a long-established commodities firm, lost $17 million after processing a series of wire transfers requested over email. The messages appeared to come from senior leadership. The tone was familiar. The urgency fit the situation. Nothing about the requests felt out of place in the moment.

The money moved. The doubt came later.

What's worth paying attention to isn't the size of the loss or the cleverness of the attack. It's that the systems involved did what they were supposed to do. The wires were valid. The approvals were followed. No one bypassed policy.

What never quite happened was certainty about who was actually on the other end of the conversation at the moment the transaction mattered.

That dynamic isn't unique. Most organizations are very good at moving money once a decision has been made. They've invested heavily in rails, controls, and workflows that govern how funds flow through the system.

What tends to get less attention is how identity is verified in the first place—especially in a digital environment.

Email and messaging systems don't carry identity the way in-person interaction used to. A message from a colleague looks the same whether it's coming from their laptop or from someone who gained access to their account. A phone call sounds convincing whether it's a trusted advisor or someone who has listened carefully enough to sound ordinary and familiar.

Unless a system is designed to ask, it can't tell the difference.

And criminals target that gap between appearance and authenticity.

When participation is based on familiarity—recognizing a name, a style, a pattern—security becomes situational. And when that fails, everything downstream inherits that failure.

This is why verification, at its core, isn't about adding more tools. It's about building proof into the moments where value moves. Proof of who is participating right now. Proof that they're authorized to give the instructions being followed. Proof that those instructions haven't been altered along the way.

That proof doesn't come from a single check. It comes from sequence and structure—placing verification where judgment is weakest and pressure is highest.

Once you start there, the other pieces of integrity fall into place.

Layering Security Over Three Points of Certainty

One of the mistakes organizations make is treating security as a collection of tools: multifactor authentication here, approval rules there, a callback procedure when something feels off. Each piece might be sensible on its own, but without a structure underneath, they don't add up to certainty.

What works better is to think in layers of certainty.

In practice, effective transaction security comes down to three distinct layers, applied in sequence: **authentication, identity verification, and account validation**. Each layer answers a different question. If you skip one, the rest inherit the gap.

The first layer is **Authentication**. This is the layer most people are familiar with. Logins. Passkeys. Single sign-on. Multifactor authentication. Device controls.

Its job is straightforward: prevent unauthorized presence.

Authentication doesn't prove who someone is in a real-world sense. It proves they're allowed to be *here*—inside the system where transactions are created, reviewed, and executed.

This matters more than it sounds.

When execution happens across email threads, spreadsheets, chat messages, and side conversations, security fragments. No single control applies consistently. Decisions get made outside the environment where verification is possible.

A single, shared execution environment creates a common playing field. Everyone involved operates under the same rules, the same visibility, and the same constraints. If the work happens elsewhere, security can't follow it.

Authentication answers the first question: *Should this person be present at all?*

Layer number two is **Identity Verification**.

Once someone is inside the environment, the next question becomes more specific: *Is this person actually who they claim to be right now?*

Identity verification exists to remove judgment from that moment. Not permanently. Not socially. But deterministically, at the points where participation matters.

Effective identity verification is layered by design. No single method is sufficient on its own.

Documentation helps establish legal existence. Database checks confirm claims against independent sources. Biometrics, where appropriate, tie access to physical presence. Out-of-band callbacks break the illusion created by compromised inboxes.

What matters isn't which methods you use. It's that verification is repeatable, non-optional, and embedded in the process.

But identity isn't just an individual. It's a network.

Executives have assistants. Vendors have controllers. Law firms have associates. Banks have teams. Transactions move through these networks, not just through named principals.

That's why identity verification has to extend to the contact networks we discussed in Chapter 8.

Organizations that do this well follow three simple disciplines:

- They verify contacts before adding them
- They treat changes as events that require confirmation through previously trusted channels
- They monitor for behavioral shifts that don't match established patterns

Still, even perfect identity verification has a limit.

You can be absolutely certain *who* you're talking to—and still send money to the wrong place.

That's where layer three comes in: **Account Validation**.

Account validation answers the question: *Does the verified participant actually control the destination they've provided?*

This distinction matters because account numbers aren't identity. They're directions.

Banks confirm that an account exists. They don't confirm that it belongs to the vendor on your invoice or the counterparty in your contract. If the numbers are valid, the money moves.

That's why so many losses happen after identity appears to be settled.

Account validation works as a stack, not a single check:

- Format validation catches errors and crude spoofs
- Out-of-band confirmation ties new instructions back to known record
- Micro-deposits or test transfers prove control before scale
- Name-matching provides signals, not truth
- Account tenure and behavior add context
- Reason verification asks a simple operational question: *Why this change, and why now?*

The most important discipline here is treating every account change as a new relationship, even when it arrives under a familiar name. That pause—built into the system rather than left to discretion—is where most fraud collapses.

These layers only work when applied in order.

Authentication limits who can enter the system. Identity verification confirms who is participating. Account validation ensures value goes where it's intended.

Skip a step, and the rest become fragile.

The most important piece of this puzzle is consistency. Verification happens the same way, every time, regardless of urgency or seniority. Deviation isn't a judgment call—it's simply not available.

Building Verification Into Your Workflow

The easiest way to weaken verification is to treat it as something people are supposed to remember to do.

Under pressure, memory loses to momentum every time.

What works is designing verification so it happens as part of the transaction itself. A good place to start is by deciding when verification is required, rather than who is responsible for it. The system shouldn't wait for someone to feel uneasy before it asks for proof.

That framing alone removes a lot of ambiguity.

First, **require verification before participation.** If a person, vendor, advisor, or counterparty is going to be involved in a transaction—initiating, approving, coordinating, or releasing—there should already be a verified identity attached to that role.

This is especially important for people who influence transactions without touching money directly. Assistants, outside counsel, and vendor contacts shape decisions long before funds move. If they can participate, they should be verified.

When verification is upstream, it stops being personal. It's just part of getting access to the work.

Second, **build and share a verified contact directory.** Most organizations already have contact information. What they often lack is a shared understanding of what's been verified and what hasn't.

A verified contact directory solves that.

It doesn't need to be elaborate. What matters is that it's shared, visible, and carries provenance.

For each contact, the organization should be able to see:

- Who added the contact
- How their identity was verified
- Which channels are approved for communication
- When verification last occurred

When this information lives in one place, people stop relying on personal address books and memory. They stop guessing which phone number is "the right one." The system provides the answer.

This also creates institutional memory. When people leave, verification doesn't leave with them.

Next, **treat change as a risk event.** Most fraud enters through change, not through creation.

New bank account. New email address. New point of contact. New urgency layered on top of something familiar.

Change deserves structure.

Any update to identity, contact details, or payment instructions should automatically trigger re-verification through previously confirmed channels. Not the ones provided in the request. The ones *already on file*.

This applies even when the relationship is long-standing. Especially then.

Treating every change as a fresh identity event removes the pressure to decide whether something "feels legitimate." The process answers that question instead.

Lastly, **let systems surface anomalies, not override judgment.** Good verification systems don't try to replace human judgment. They support it.

The goal isn't just to blindly block everything unusual. It's to make unusual things visible.

Behavioral shifts, timing changes, new destinations, or deviations from past patterns should be surfaced clearly and early. What happens next can be a human decision—but it should be an informed one.

Equally important, the system should never force someone to proceed just because the boxes are checked. If something doesn't make sense, stopping should be easy and legitimate.

Verification works best when people feel supported in slowing things down, not blamed for it.

The result of these small workflow changes? A verification process that's predictable and repeatable.

When Verification Becomes Routine

When verification is built into the workflow, the experience of doing the work changes in quiet but meaningful ways.

People no longer carry the burden of deciding whether a message, request, or document feels trustworthy in the moment. The system already knows what has been verified, what hasn't, and what needs confirmation before anything can proceed. That clarity shows up early, before urgency has a chance to take over.

Unusual requests surface naturally. A new account without prior verification pauses where it should. A change that doesn't match past behavior becomes visible without anyone needing to raise their voice. Timing, destination, and participation all line up against what the system already understands to be valid.

For the people inside the organization, this creates a sense of protection.

Pausing a transaction feels normal. Calling back feels routine. Asking for a second factor doesn't require justification. The process provides the reason. Individuals don't have to invent explanations or worry about how their caution will be perceived.

Over time, this becomes the rhythm of work.

Verified directories get used instead of personal contact lists. Out-of-band confirmation becomes muscle memory. Delays caused by verification feel predictable and contained rather than disruptive. Real partners adapt quickly, because verification fits naturally into how legitimate relationships operate.

Trust deepens in a practical way. Teams trust the information in front of them because it has been proven. Leaders trust outcomes because they know the inputs were sound. External partners trust the organization because the rules are clear and consistent.

Security settles into the background. It no longer demands attention or special handling. It shows up as a set of rails that guide transactions forward and quietly prevent the ones that don't belong.

That's the outcome of getting verification right. Not heightened vigilance, but durable certainty.

CHAPTER 11: Information

Bringing data under the same control plane

In 1937, a museum in Rotterdam paid a fortune for a newly discovered painting attributed to Johannes Vermeer.

The brushwork was convincing. The pigments matched the period. The subject fit Vermeer's known body of work.

Most importantly, it was authenticated by Abraham Bredius, one of the world's leading experts on Dutch masters. Bredius didn't hedge. He declared it a masterpiece. The painting entered the collection of the Rotterdam Museum as a genuine Vermeer.

It wasn't.

The painting was the work of Han van Meegeren, a skilled forger who had spent years studying Vermeer's technique. He mixed his own pigments using period-accurate materials. He baked canvases to simulate age. He learned how to make his work look right—convincingly, confidently right.

And that was the problem.

The painting passed every test that relied on appearance and expertise. What it lacked was something quieter and harder to fake: provenance. There was no unbroken chain of custody. No clear, documented history tracing the painting from its creation to the present day.

Once that absence was understood, the illusion collapsed.

The art world learned a hard lesson from van Meegeren and the forgers who followed him. Authenticity is not a matter of how something looks. It's a matter of where it's been, who touched it, and whether that story holds together over time.

Without provenance, a painting is just pigment and canvas.

Transaction documents live in the same tension. A wire instruction can look right. A closing memo can feel routine. A PDF can carry the right logos, the right names, and the right tone.

But appearance isn't proof.

If you can't trace where a document came from, how it entered the transaction, and whether it's remained intact since, then it's just text and formatting—no matter how official it appears.

That's the problem we're going to solve in this chapter.

Why Transaction Documentation is Structurally Vulnerable

If identity requires verification, documentation requires provenance. That's the simplest way to understand the problem.

Most organizations already accept that you can't rely on appearance when it comes to people. A familiar name in an email signature doesn't prove who's actually on the other end. That's why we verify identity, assign roles, and restrict authority (like we discussed in the previous chapters).

But when it comes to documentation—the instructions, authorizations, and records that tell the system what to do—we quietly drop that same discipline.

We treat documents as passive objects instead of active controls.

The assumption is that if it came from the right place, at roughly the right time, and looks the way it's supposed to look, then it must be legitimate. But that's not a safe assumption.

Here's the core problem: In most real transactions, information lives everywhere.

Wire instructions might exist as:

- An email attachment from three weeks ago
- A revised PDF forwarded on Friday afternoon
- A screenshot pasted into a text message
- A version saved to someone's desktop "just in case"

Invoices get downloaded, renamed, reattached, and forwarded. Closing documents get revised in parallel by multiple parties. Account updates arrive as "quick changes" late in the process.

No one intends to create confusion. It's just how work flows.

But in that flow, documentation quietly loses its chain of custody. It's how you end up sitting at your desk on a Friday afternoon looking at four different versions of the same wire instructions, trying to figure out which one is correct.

Once that happens, authenticity becomes guesswork. And guesswork is where fraud, error, and delay thrive.

To better understand this vulnerability—and how to fix it—it helps to look at documentation the same way we look at money movement.

There are three moments when things break.

The Three Vectors of Information Risk

Every transaction document is exposed in three distinct ways:

1. **Obtaining:** How the information enters the transaction
2. **Transit:** How it moves between the participants
3. **Storage:** Where it lives over time

Each vector creates a different kind of risk. And each requires a different kind of control.

Vector 1: Obtaining

Authenticity has to exist at the point of entry.

If a document isn't verified when it first enters the transaction, there is no reliable way to secure it later. Everything that follows is built on an assumption instead of evidence.

We already know how to do this well—we just don't apply it consistently.

Consider how high-stakes municipal bond documents are created. These deals require multiple layers of authentication before a single dollar moves:

- **Institutional seals** establish that the organization itself has authorized the document
- **Individual signatures** confirm that people with actual authority approved it
- **Notary attestations** verify identity, physical presence, and timing

That combination creates provenance at the moment of origin. There is a record of who acted, when they acted, and that an independent party confirmed it happened as claimed.

The document doesn't become trusted later. It is born trusted.

Now contrast that with how most transaction documentation enters organizations today. Wire instructions arrive as PDFs attached to emails. Invoices are forwarded through long chains. Account changes appear as "updated information" with no verification of how they were created or who actually authorized them. They look legitimate. But nothing about their origin has been authenticated.

That's the gap.

The stakes in a bond deal justify rigor, but the principle applies everywhere. If a document governs the movement of money, authority, or obligation, then it deserves provenance at entry—regardless of size.

This is where digitization creates opportunity, not weakness. Done correctly, digital signature processes can be even stronger than physical ones. Identity verification, timestamps, and location data can be embedded directly into the act of signing. Immutable audit trails can be created automatically.

But only if verification is part of collection—not something added afterward.

You can't secure a document later if you didn't know where it came from in the first place.

Vector 2: Information in Transit

Just like money, documents are most vulnerable when they're moving.

And yet, most transaction documentation still moves through email—a system designed for conversation, not custody.

Email doesn't verify recipients. It doesn't confirm documents arrived unaltered. It doesn't provide access controls once something is sent. It doesn't create an audit trail of who viewed, downloaded, forwarded, or modified an attachment.

Once you click send, the document leaves your control entirely.

That's not a flaw. It's just not what email was built to do.

Modern fraud exploits this gap. Attackers don't invent documents from scratch. They watch real transactions unfold. They intercept legitimate communications, make small changes—a routing number, an account name—and insert those documents back into trusted workflows.

Everything looks right because *almost* everything is right.

The industry has done a good job training people to be skeptical of identity. We teach call-backs. Known numbers. Out-of-band verification.

We haven't trained the same skepticism for documentation.

A document that looks familiar, arrives at the expected time, and references the right deal often gets processed without question. Appearance becomes authentication.

That's exactly what attackers rely on.

Regulations help, but they don't solve this end-to-end. Encryption rules, retention requirements, and compliance standards address pieces of the problem. None of them establish continuous custody for transaction documents as they move between parties.

Without that, transit remains the most exposed moment in the lifecycle.

Vector 3: Storage

Even when documents enter cleanly and move without incident, they still face one more risk.

They scatter.

Before documentation can be systematically preserved, it spreads across inboxes, shared drives, local folders, and third-party systems. Multiple versions emerge. Access becomes broad by default. Context fades.

At that point, basic questions become hard to answer:

- Which version is authoritative?
- Who has access—and who shouldn't?
- Can you prove a document hasn't been altered?
- Can you show what it said at the moment a decision was made?

Most organizations have strong access controls around financial systems. Far fewer apply the same discipline to document storage.

Someone with email access often has access to years of transaction documentation. Departing employees, compromised accounts, and insider threats all create risk long after a deal is done.

And when something goes wrong—during a dispute, an audit, or an investigation—documentation becomes the evidence. If you can't demonstrate integrity, you're left reconstructing memory instead of presenting proof.

That's not a failure of people. It's a failure of structure.

Taken together, these three vectors explain why transaction documentation is so often the weakest link: Because information has been allowed to move independently of verification, roles, and control.

So, what does secure information handling look like in real transactions?

What Secure Information Handling Looks Like in Practice

Now that we've talked about the theory, let's look at the practical steps. The idea here is to add structure in the same places you already rely on trust, habit, and timing—and replacing those soft assumptions with systems that hold up under pressure.

When we obtain a document, we need to know where it came from, and whether it's important enough to require heavier verification. This entry point is arguably the most important moment in a document's lifetime, because everything downstream depends on what we do in this moment.

So, here's what you can do:

- **Define what requires provenance:** Identify documents that authorize payments, change destinations, finalize obligations, or explain disbursement logic. These documents require verified entry.
- **Use authenticated intake channels:** High-stakes documents are submitted through controlled systems where identity and authority are confirmed before acceptance.
- **Create provenance at submission:** Capture who submitted the document, when it was submitted, and under what authority. The audit trail begins immediately.

For example, in a bond deal or large real estate closing, wire instructions are defined early as documents requiring provenance. The transaction specifies how they enter the system. Instructions are submitted through a verified channel tied directly to the deal. The submitting party is known. Their authority is already mapped. The document becomes part of the transaction record the moment it enters.

If instructions need to change, the update follows the same path. A new version is submitted, linked to the prior version, and attributed clearly. Everyone can see the sequence. No one is sorting through inboxes or asking which attachment is current.

This approach reduces friction later. When documents enter cleanly, the transaction carries clarity forward instead of uncertainty. Teams spend less time double-checking and more time executing.

Next, when a document is in transit, it requires controlled movement. Transit is where verification must stay attached to the document.

Here are some ways to keep documents secure in transit:

- **Restrict informal movement of authoritative documents:** Critical documents do not circulate as open attachments or forwarded files.

- **Verify receipt and integrity:** The system confirms that documents arrive intact and are accessed by the right parties.
- **Standardize change handling:** Updates follow defined procedures that preserve version history and attribution.

Late-stage document changes are common. Without structure, they arrive as familiar-looking emails at busy moments. With structure, they trigger a defined process. The request is verified. The update is submitted through the transaction system. The prior version remains visible. The new version becomes authoritative only after confirmation.

Finally, when documentation is stored, transaction documents require one authoritative home. That home needs to preserve context, access, and history.

Here's what you can do:

- **Designate an authoritative repository:** One system holds the definitive version of each transaction document.
- **Apply role-based access and logging:** Viewing, editing, and approvals are governed by role and recorded automatically.
- **Bind documents to the transaction:** Documents inherit permissions, scope, and authority from the transaction they belong to.

With one source of truth for documentation, all participants can reference the same document record. There's no parallel set of final versions, because the system already defines which version governs the transaction.

And when the documentation is bound to a transaction, it prevents reuse outside the original context. A wire instruction tied to a specific deal cannot quietly resurface later with implied authority. The document retains its meaning because its context is preserved.

When information is handled this way, transactions run with less tension and more confidence. The system carries the load, and people are free to focus on moving the deal forward.

What You Gain With Secured Information

When information is secured properly, documentation stops being a source of uncertainty.

There's one version that matters. Everyone knows where to find it. Changes are visible, attributable, and expected instead of surprising. The quiet friction that usually surrounds transactions—checking emails, comparing attachments, asking "is this the latest?"—simply disappears.

That clarity has a direct operational payoff. Fewer delays. Fewer last-minute questions. Fewer situations where good people are forced to guess under time pressure. Documentation becomes reliable infrastructure instead of something that has to be managed manually.

Just as important, intent becomes durable. Months or years later, the transaction still makes sense. The record explains itself. Audits are faster. Disputes are shorter. Institutional memory doesn't depend on who happens to still be around.

Once information is stable, the transaction itself becomes predictable. And when a transaction is predictable, you gain something that's easy to overlook: the ability to choose *how* money should move.

That's the next layer. Different payment rails carry different tradeoffs—speed, reversibility, visibility, recovery. Those choices only matter once the information guiding them is trustworthy.

With information under control, we can finally talk about how money should move—and why.

CHAPTER 12: Payment Rails

The rail you choose is the risk you accept

The deal is done. Documents are signed, or about to be. Everyone's tired, but relieved. Lawyers are watching the clock. The counterparty wants confirmation. Someone finally asks the question that sounds purely logistical:

"How should we send the money?"

The answer is framed in the language everyone knows: cost and speed. Wire is fast, but expensive. ACH is cheap, but slow. Same-day ACH feels like the right middle ground.

In most cases, teams don't frame this as a security decision. It's treated as execution—the last small step after all the "real work" is finished. So, the finance lead makes a sensible call: Same-day ACH. Fast enough to keep things moving. Cheap enough that no one complains. Familiar enough that nobody hesitates.

Nothing about that choice feels reckless. But what's easy to miss in that moment is that this decision quietly determines the *environment* the transaction is about to enter.

That environment is what this chapter is about.

Transactions are sometimes put at a much higher risk because they're being run directly against vaults. A core operating account is meant to hold value. It's where reserves sit. It's where stability lives. It's not designed for decision-making, pressure, or last-minute change.

And yet, most organizations still execute transactions straight from that account.

Wires originate directly from it. ACH pulls directly from it. Credentials that can move money are pointed right at reserves.

Under normal conditions, this mostly works. Under pressure, it doesn't.

Security doesn't come from stronger vaults. It comes from **distance**.

In the physical world, we already understand this. We don't run hot water and cold water through the same pipe. We don't connect toilets directly to water heaters. We layer plumbing because different actions carry different risks. That prevents one failure from contaminating everything else.

Banking should work the same way.

Transactions are dynamic. Vaults are static. Treating them as the same thing is where things start to break.

A transaction layer sits between intent and execution. It's the space where verification can happen without touching reserves. Where approvals have room to mean something. Where pressure doesn't automatically translate into irreversible movement.

When that separation layer exists, payment rails become tools. When it doesn't, every rail becomes dangerous, because it's directly connected to the vault.

So when the finance lead under pressure chooses same-day ACH at the last minute, it's not necessarily the wrong rail. But there's a deeper architectural problem underneath.

By the time execution happened, the most important decision had already been made—and it wasn't named as a security decision at all. The rail was plugged straight into reserves. There was no buffer. No containment. No separation between approval and loss.

That's why criminals don't just target people. They target environments. They look for moments where time, familiarity, and pressure collapse verification. They don't need malware or clever tricks. They just need a system that assumes execution is "just plumbing."

Plumbing determines flow. And in finance, flow determines risk.

This chapter isn't about telling you which rail is good or bad. It's about helping you see rail choice for what it actually is: a security architecture decision that usually gets made under the wrong framing.

Once you see that clearly, choosing the right rail falls into place.

How Rails Shape Risk

Once transactions are properly layered away from the vault, rail choice starts to matter in a different way.

At that point, the rail doesn't determine *whether* a transaction is safe. It determines how risk behaves once money starts moving.

Every payment rail creates a different security environment. Not because one is good and another is bad, but because each one answers four practical questions differently:

- How much time do you have to verify?
- Can anyone intervene once the process starts?
- What does recovery look like if something goes wrong?

- What evidence exists after the fact?

Those are the differences that criminals study.

That's why it's helpful to stop thinking about rails as mere delivery mechanisms and start thinking about them as **operating environments**. The rails themselves are neutral. What makes them dangerous is context collapse—when speed, familiarity, and pressure compress visibility at exactly the wrong moment.

Criminals don't need to trick people if they can pick the environment where verification is least likely to survive.

Let's walk through the major rails with that lens.

Wire Transfers

Wire transfers were built for immediacy. They settle quickly, and they're irrevocable by design. Once funds leave, there's no built-in recovery mechanism.

That finality is the point.

In markets where certainty matters, wires are a feature, not a flaw. When everyone is properly authenticated and the context is clear, irreversibility becomes an asset. No clawbacks. No ambiguity. The transaction is done.

The security implication is straightforward: **all meaningful controls must exist before the send**.

If a wire is originating directly from the vault, there is no second chance. Verification, authorization, and intent confirmation have to be completed before execution. Once the button is pressed, the system does exactly what it's supposed to do.

This makes wires well-suited for vetted, high-value, one-time closings where parties are known, documentation is complete, and last-minute changes are unlikely.

They're poorly suited for routine payments or situations where instructions might shift late in the process. That's where familiarity and muscle memory can quietly override scrutiny.

Speed doesn't make wires inherently dangerous. They're only dangerous when speed is combined with assumptions.

ACH (Standard)

Standard ACH lives at the opposite end of the spectrum. It's batch-based. Settlement is delayed. There's a built-in window for reversals and returns.

ACH was designed for reliability, not urgency. Payroll, tuition, vendor batches—this system assumes predictability and repetition. Time is part of the design.

When ACH is layered properly away from the vault, that delay acts as a buffer. It creates space for reconciliation, anomaly detection, and intervention before loss becomes permanent.

That makes ACH a strong fit for predictable, recurring payments where patterns are stable and deviations are easy to spot.

But delay cuts both ways. Criminals know ACH doesn't demand immediate attention. They exploit reconciliation noise. Small fraudulent entries can hide in volume. Confusion created by returns and adjustments can mask intent.

The security of ACH depends on whether the organization uses the time it provides.

Same-Day ACH

Same-day ACH sits in an uncomfortable middle ground. It's faster than standard ACH, and it still feels reversible. But the recovery window is much smaller.

I find that perception and reality tend to diverge here.

Same-day ACH feels safer than a wire because it's "not final." But under pressure, it behaves much more like one. Verification time shrinks. Awareness still lags. By the time someone notices something is off, options are limited.

This is why criminals like it.

Same-day ACH collapses verification time without triggering the psychological caution that wires tend to provoke. It feels operational, not momentous. That familiarity is useful to attackers.

Organizations underestimate it for the same reason. It doesn't carry the emotional weight of a wire, so it doesn't always get wire-level scrutiny—even though, in practice, it often deserves it.

Again, that doesn't mean it's always the wrong choice. But you need to treat it honestly.

Real-Time Payments

Real-time payment systems like RTP or FedNow change the equation again. Settlement is instant, and there's no post-send intervention. Once funds move, they're gone.

Is this payment rail reckless? Not necessarily. But it is unforgiving.

In real-time systems, controls cannot live with people. There's no room for phone calls, second looks, or escalation after execution. Oversight has to live inside the transaction layer itself.

Banks and investigators have started using a term for what happens when this isn't true: *fraud velocity*. Losses accumulate faster than alerts can refresh. The system outruns human reflexes.

RTP works best when transactions are highly constrained, well-understood, and monitored automatically. When it's used thoughtfully, it can be incredibly effective. When it's layered poorly, it leaves no margin for correction.

Checks

Yes, checks are still with us. Mainly because people got used to them, and some organizations still prefer the manual process.

But they include a lot of inherent dangers.

Checks introduce human exposure. Mail interception, alteration, and impersonation risks are real. I'd say it's one of the most dangerous forms of payment in existence. And most of the fraud I've seen in the last twenty years has been check fraud.

While it can have benefits from being a familiar option, this is one I'd recommend to stay away from.

The takeaway from this discussion is that **rail choice determines how risk unfolds once money moves**.

Once transactions are layered away from the vault, this becomes a strategic advantage. Rail choice becomes a way to shape verification time, intervention opportunities, recovery paths, and evidence quality—intentionally, not by habit.

That's the mindset shift.

From here, the question isn't "Which rail is cheapest or fastest?"

It's "Which rail creates the right environment for this transaction?"

When you ask that question, you allow execution to reinforce security instead of undermining it.

Making Rail Choice Operational

Once you start seeing payment rails as security environments, the next step is to improve the structure around those rails.

Our goal here is to make sure execution reinforces the work you've already done upstream—rather than quietly undoing it at the last minute.

That starts with **making rail choice an intentional decision**.

Execution is the final security gate, because it's the last irreversible step in a transaction.

When transactions are properly layered away from the vault, execution becomes the point where all prior verification proves itself. Identity checks, account validation, approvals, staging—none of that matters if the final movement ignores the environment it's entering.

Rail choice should never be an afterthought tagged onto the end of a transaction. It deserves the same intentionality as approval authority or transaction classification.

The key distinction is this: execution only works as a security gate when the vault is no longer exposed. Once reserves are protected by a transaction layer, rail choice becomes a way to *shape risk*, not absorb it blindly.

Verification discipline has to carry through execution—not stop just before it.

Next, **match the rail to the transaction type**.

Once rail choice is explicit, patterns become easier to see:

- **High-value, one-time closings** between vetted parties often justify a wire. In those cases, maximum verification up front makes irreversibility an asset.
- In **routine vendor payments**, predictability matters more than speed. ACH fits well here because reversibility and pattern analysis provide a safety net, especially when transactions repeat regularly.
- **Emergency payments** shouldn't rely on ad-hoc decisions made under pressure. The safest organizations pre-approve fast paths with defined rails, limits, and controls—so urgency doesn't bypass verification.

New rails deserve restraint. Whether it's RTP, FedNow, or something emerging, the discipline is the same: start with low-risk flows. Learn the patterns. Understand how detection and recovery actually behave before expanding use.

To keep matching the right rails to the right payments, **build a payment rail portfolio**.

Mature treasury teams don't pick a favorite rail. They build a portfolio.

Different rails serve different risk profiles. A half-million-dollar wire is not the same risk event as a half-million-dollar ACH. Amount alone doesn't tell the story. Environment does.

Most organizations tack on additional reviews based on the amount of money that's moving, not the payment rail. That can make sense, but it's important to remember that different payment rails typically mean different purposes. So when deciding which transactions need more review, it's important to set dollar thresholds based on the payment rail. The same amount moving through different systems creates very different security conditions.

When rail choice becomes part of transaction classification, execution stops being reactive. The system already knows what kind of scrutiny, approval, and monitoring applies—before anyone feels rushed.

This portfolio mindset also reframes cost conversations. Wire fees are visible. Fraud losses are abstract until they're not. Optimizing for the cheapest rail without considering the security environment is a false economy. You can save fees a hundred times and still lose everything once.

Security architecture doesn't always show up on a budget line. But it prevents outcomes that dominate budgets for years.

Finally, **make your rail selection visible**.

One of the simplest—and most powerful—changes organizations can make is documenting *why* a rail was chosen. Make reasoning explicit.

When auditors, insurers, or investigators can see not just that a payment was sent, but why that rail made sense for that transaction, the organization's security posture becomes legible. Evidence exists because structure exists.

That philosophy is starting to influence policy. After a string of multimillion-dollar business email compromise cases, the Federal Reserve and NACHA both urged financial institutions to implement "Know Your Payment Flow" frameworks—essentially mapping who uses which rails for what purpose and why.

Fraud thrives in opacity. It struggles in environments where decisions leave a trail.

Making rail selection visible also sends a signal—to internal teams and to external observers—that execution is intentional. Over time, that signal compounds. Criminals learn which environments are designed and which ones default to convenience.

The practical takeaway here is simple: rail choice shouldn't be implicit. It should be explainable.

When Execution Stops Undoing Your Work

When rail choice is treated as architecture instead of logistics, execution starts doing its job.

It reinforces security instead of quietly undoing it.

The final movement of money no longer depends on habit, memory, or last-minute judgment calls. It follows a structure that already accounts for time pressure, irreversibility, and recovery. Execution becomes confirmation, not a gamble.

Criminals respond to this immediately.

They look for environments where speed collapses visibility and where rail choice changes based on convenience. When those patterns aren't there—when execution is deliberate and predictable—they tend to move on. Not because fraud is impossible, but because it's inefficient. Designed systems are harder to exploit than improvised ones.

Teams feel the difference as well.

Work moves faster because fear drops out of the process. People don't slow down to double-guess themselves at the last moment. The system already knows what's allowed, what's risky, and what requires more scrutiny. There's less escalation, fewer exceptions, and far fewer "are we sure?" moments right before money moves.

Rail choice, in this environment, becomes a visible signal of maturity.

Not because of which rail is used, but because the choice makes sense. There's an architecture underneath it. Anyone looking closely can see that execution isn't ad hoc—it's intentional.

You don't need a new bank. You don't need new money. You don't need exotic rails.

You need a transaction layer. And then you need to choose rails like an architect, not a courier. Execution is where security proves itself. The rail you choose is the architecture you trust.

Of course, even when you design your transactions well, there's never a guarantee that your organization is protected from fraud. So, let's talk about what you can do when things go wrong.

CHAPTER 13: Transaction Insurance

Insurance doesn't prevent fraud—it proves you tried to prevent it

---

In 2016, Apache Corporation lost $7 million to criminals who had patiently inserted themselves into the company's normal vendor-payment workflow.

The attack wasn't flashy. Phone calls that sounded legitimate. Emails that matched prior correspondence. Forged letterhead that aligned with what employees expected to see. The people involved believed they were following reasonable verification steps. From the inside, it looked like routine work under time pressure.

When Apache filed an insurance claim, they expected recovery.

They didn't get it.

The case eventually made its way to the Fifth Circuit Court of Appeals, where the judges focused on a distinction that still surprises many finance teams. The court ruled that the loss wasn't covered because the fraud was "merely incidental" to what was, in legal terms, a voluntary and authorized transfer. Apache hadn't had money *taken* from them. They had *sent* it.

That distinction decided the case.

Not long before that ruling, a remarkably similar fraud had played out in Michigan. American Tooling Center lost $834,000 after criminals intercepted communications with a legitimate Chinese supplier and substituted fraudulent wire instructions. Believing the emails were genuine, the finance team sent payment to accounts controlled by criminals.

Their insurer initially denied the claim using logic similar to Apache's case.

American Tooling pushed back.

When the dispute reached the Sixth Circuit, the outcome flipped. The court ruled in American Tooling's favor, holding that the loss was covered under the policy's computer fraud provision. The reason was subtle but decisive: the fraud depended on deceptive emails to succeed. Those emails weren't background noise. They were the mechanism of the theft.

Two companies. Nearly identical attacks. Very different outcomes.

An underwriter once explained the business to me with a pencil and two boxes. In the first, he wrote "Losses caused by criminals." In the second, "Losses caused by you." Then he drew an arrow between them. "The debate," he said, "is where that arrow lands." If criminals fool you into sending money, did they take it, or did you give it away? Your answer determines whether the insurer pays or wishes you well.

When American Tooling reconstructed what happened, they were able to show that their team had followed established verification procedures. Those procedures were consistent, documented, and designed to prevent exactly this kind of loss. The fraud succeeded not because someone skipped steps, but because criminals deliberately studied the workflow and defeated it.

In Apache's case, the court didn't see the same level of systematic process. So, the loss was characterized as an internal failure rather than an external crime.

The difference? Evidence.

When insurance is built into the transaction processes, it doesn't slow teams down or make them afraid to act. It gives them the confidence to move money knowing that disciplined execution will be backed up if professional deception succeeds anyway.

That's the role insurance plays in a modern transaction system.

What Transaction Insurance Actually Insures

Every transaction creates liability.

The moment money moves, responsibility attaches to the outcome. That's true whether the transaction is routine or rare, small or consequential. What changes is not whether liability exists, but where it ultimately lands.

In practice, liability settles in one of three places:

- On an individual
- On the organization
- Or on an insurance policy

Transaction insurance exists to move that burden away from people and into a structure designed to absorb it. But that transfer only happens under specific conditions.

Insurance does not absorb loss automatically. It absorbs loss when the organization can demonstrate that the transaction occurred inside a disciplined system.

From an insurer's perspective, coverage depends on three facts that have to line up.

1. **First, systematic controls existed before the transaction occurred**. Clear procedures for approving, verifying, and releasing funds. Defined roles. Expected checks.
2. **Second, those controls were followed in the transaction that failed**. Not generally, not aspirationally, but in that specific moment. The process ran the way it was designed to run.
3. **Third, fraud succeeded anyway**.

When all three are true, insurance can do its job. The loss is understood as the result of professional deception defeating a reasonable system, rather than the result of unmanaged discretion.

That distinction drives nearly every claim outcome.

Courts and carriers draw a sharp line between two kinds of loss: Money taken from you, and money you voluntarily sent

From a human perspective, both feel the same. From an insurance perspective, they are very different events.

When money is taken through unauthorized system access, insurers tend to view the loss as external theft. When money is sent by an authorized employee acting on fraudulent instructions, insurers scrutinize the process that led to the transfer.

This is where documentation, controls, and consistency begin to matter more than intent.

The different outcomes for Apache Corporation and American Tooling weren't because of the sophistication of the criminals involved.

Simply put, one company could demonstrate disciplined execution. The other could not.

Why Policy Language Deserves Special Attention

Transaction insurance is heavily dependent on wording. Small phrases carry large consequences.

Most crime policies are built from several distinct coverage categories, each with different triggers and limitations:

- **Computer Fraud** typically requires unauthorized access to your systems

- **Funds Transfer Fraud** often ties coverage to fraudulent instructions purporting to come from a bank
- **Social Engineering or Fraudulent Instruction** addresses deception that induces authorized employees to send money

Each category covers a different failure mode. Each has its own exclusions, conditions, and sublimits.

The words used to link cause and loss matter. Some policies are written so narrowly that coverage breaks the moment an employee takes an authorized action. Broader language keeps the focus on the deception itself, even when the final step is carried out by someone acting in good faith.

Another practical reality is limits. Social engineering coverage is often capped far below the headline crime limit. Organizations sometimes discover this only after a loss, when the available coverage is misaligned with the size of real-world transactions.

These outcomes are rarely surprises at claim time. They are consequences of how the policy was negotiated.

Coverage gaps, sublimits, and restrictive conditions are usually visible in the policy language. They persist because organizations focus on price, not on how the policy interacts with actual transaction behavior.

Insurers are clear about what they will support. They price based on evidence. When controls are documented, followed, and tested, terms improve. When they aren't, exclusions and tight limits fill the space.

This is why insurance works best when it is considered alongside transaction design, not after.

How to Earn Coverage Before You Need It

Arguing your case well isn't a guarantee you'll get the coverage you need. Claims succeed because the organization built evidence into everyday work, long before anything went wrong.

Of course, you can't predict when fraud may happen in your organization. What you *can* do is create systems that are observable. So when fraud happens, the story tells itself.

In basic terms, that means putting into practice everything we've covered so far in this book.

Here's how to do it specifically:

First, **build the process out in advance.** Think of this as building the claim file before a claim exists.

Start with plain-language procedures. Not policy manuals written for auditors, but clear descriptions of how transactions actually move: who initiates, who verifies, who approves, and who releases funds. If someone new joins the team tomorrow, they should be able to follow the process without tribal knowledge.

Every verification step should leave a trace.

- Callbacks logged to known numbers
- Notes showing what was verified and why it was acceptable
- Approvals recorded with names, timestamps, and scope
- Release workflows that prove no one person can move money end-to-end

Segregation of duties matters less as an idea than as proof. Initiator, approver, and releaser should be visibly distinct in the record. When those roles collapse into one person, even occasionally, the exception should be documented with a reason and an approval.

Having this documented is all about having clarity when it matters most.

Second, **make controls observable**. Controls that exist only in people's heads don't count when it matters.

If a control is important enough to rely on, it's important enough to be visible after the fact. That means designing workflows so evidence is created automatically as work happens.

- Systems that log who did what and when
- Directories that show how vendors were onboarded and last verified
- Release histories that show consistency, not improvisation

Observable controls reduce debate. They make it easier for an outside party to understand how your organization operates without reading intent into silence.

Next, **create transaction evidence by default**. Strong teams don't reconstruct history under stress. They let their systems speak.

Transaction evidence should accumulate quietly as part of normal execution. When that happens, fraud investigations and insurance claims stop being exercises in memory and become exercises in review.

All this preparation puts you in the best position to fight back when fraud happens.

What to Do in the Moment Fraud Happens

Even when your organization is prepared, facing fraud still feels like a punch to the gut. In that moment, it's important that everyone on the team understands what to do next. Then, you can move forward toward resolving the issue without hesitation or fear.

Here are four clear steps to take:

- **Prepare for the claim moment immediately:** Time matters as soon as fraud is discovered. Notify your bank right away to request recall attempts and account freezes, file a law enforcement report with real detail, and place a preservation hold on relevant systems so logs and records remain intact. These early actions support recovery and protect the integrity of your eventual claim.
- **Designate a claims lead:** One person should own coordination from the start. This role keeps timelines straight, gathers documentation, and ensures decisions are made deliberately rather than reactively. Clear ownership prevents confusion at the exact moment clarity matters most.
- **Centralize communication with the insurer:** All communication with the carrier should flow through the claims lead. This keeps the story consistent, avoids fragmented explanations, and reduces the risk of well-intentioned but incomplete responses creating problems later. It also allows the rest of the team to stay focused on stabilizing operations.
- **Negotiate with receipts:** The strongest claims are built on evidence, not urgency. Share logs, approval histories, vendor records, and documented exceptions that show how the transaction was executed. When the record is clear, conversations move quickly from questioning what happened to resolving how the claim will be paid.

The most effective recoveries follow a clear, practiced sequence that doesn't depend on who happens to be in the office. When gathering evidence is just part of the process, and your team is aware of what they need to do in case of fraud, insurance becomes what it's meant to be: a quiet backstop that lets your people keep doing their jobs with confidence, even when something goes wrong.

The Real Goal of Transaction Insurance: Protecting People

The most important function of transaction insurance is not financial recovery. It is human protection.

When loss transfers to a policy, individuals are not left personally carrying the weight of a sophisticated crime. Teams do not freeze after a mistake. Organizations retain their ability to move money without fear becoming the dominant control.

That stability has real consequences.

Organizations with credible insurance structures recruit better. Talented finance professionals look for environments where disciplined execution is supported, not punished. Decision-making improves when people know that following the system protects them even when deception succeeds.

Insurance, used this way, becomes a stabilizer. It absorbs shock so momentum survives.

It allows organizations to acknowledge a hard truth: competent people, working inside reasonable systems, can still be deceived. When that happens, the system should respond with support, not blame.

That is what transaction insurance is meant to insure.

You've Learned the Framework: Here's What's Next

Everything we've covered so far in the Secure Transactions Framework—how sound decisions are made, how governance creates clarity and alignment, and how integrity carries through execution—forms a strong foundation for modern transactions. When these elements are designed together, money moves with intention and consistency. Teams know what they're responsible for. Systems know how to respond. Execution becomes a natural extension of the work that came before it.

This foundation does more than protect transactions; it supports progress. With it in place, organizations can move forward confidently, knowing their transaction systems are built to hold up under pressure and evolve over time. In the next part of the book, we'll look beyond protecting transactions as they exist now and focus on how to build systems that are ready for what's coming—systems that can adapt, scale, and keep working even as the world around them changes.

Part Three

Building for What's Next

CHAPTER 14: Future-Proofing Your Transactions

Technology amplifies what structure protects

Remember what it was like to do big transactions in the early 90's? Don't think too hard about the changes in technology and payment rails, you might get whiplash.

And yet, as much as technology has advanced, the work hasn't changed. People still have to decide. Approve. Execute. Move money with intent.

That can create a scary reality for modern finance teams.

In early 2024, a finance officer joined what looked like a normal video meeting. The CFO talked through an urgent transfer to wrap up a deal. Colleagues nodded along in tidy little squares. The request was urgent but plausible, the kind that shows up late in a deal when everyone just wants to finish cleanly.

The request went through within minutes: a handful of high-value transfers—about $25 million in total—out the door and gone.

Only later did Arup, the British engineering firm behind the Sydney Opera House and Beijing's Bird's Nest stadium, learn that the meeting had been totally synthetic. The executives weren't there. The voices were cloned. The whole meeting was a movie staged in real time.

That story tends to get told as a warning about AI.

What's happening right now can feel like a sudden break from the past. New tools. New threats. New headlines. It's easy to assume we've crossed into entirely new territory and that whatever worked before won't hold up anymore. And it can make you feel like the decisions you make—or even your career in finance—is built on shaky ground.

But that's not actually what's happening.

AI didn't create new vulnerabilities in transactions. It accelerated the ones that were already there. It took gaps that existed quietly—reliance on familiarity, compressed decision-making, collapsed verification—and made them show up faster and more convincingly.

That's important, because it means the problem hasn't changed shape.

And when the problem doesn't change shape, the solution doesn't need to either.

Everything we've discussed so far in this book was designed around a simple idea: transactions should rely on proof, not perception. Evidence, not intuition. Structure, not heroics. That philosophy doesn't weaken in a world of synthetic media. It gets stronger.

When voices can be cloned and faces can be replicated, any system based on recognition is going to struggle. Systems based on verification were built for exactly this moment, even if we didn't realize it at the time.

That's the calming part.

You don't need a new framework for the future. You need to finish implementing the one you already have. Fully. Consistently. Without leaving the last steps to habit or trust.

The work ahead isn't about reacting to AI. It's about completing a system that was meant to outlast any single technology shift. Once that system is in place, change stops feeling existential and starts feeling manageable.

In this chapter, we'll talk about how to future-proof transactions in two practical ways:

- First, how the Secure Transactions Framework absorbs more advanced, automated, and synthetic fraud without needing to be reinvented.
- Second, how the same framework holds when new payment rails and technologies show up underneath it.
- And third, how a new kind of participant is beginning to change what preparation looks like before any of that execution begins.

The goal isn't to outpace change. It's to build systems that stay solid even as everything around them moves faster.

Future-Proofing Area #1: Automated and Synthetic Fraud

Synthetic media has crossed a practical threshold. Voices, faces, and writing styles no longer require rare skill or specialized equipment. A few minutes of source material and ordinary tools are enough to produce something convincing. Recognition—something we leaned on quietly for decades—no longer carries the weight it once did.

That sounds dramatic when stated plainly. In practice, it's simply a shift in inputs.

People still ask for money. Transactions still have timing pressure. Authority still matters. What changed is that familiarity can be performed cheaply.

This is where well-designed systems show their strength. The framework doesn't depend on recognizing who someone appears to be. It depends on how a transaction is structured end to end.

Classification does the first piece of work. High-risk transactions are identified early and treated differently by design. The amount, the timing, the irreversibility, and the context determine the controls. Who is asking doesn't change the classification. That removes discretion from the moment when pressure is highest.

Transaction staging spreads verification across time. Instead of everything hinging on a single call, meeting, or message, evidence accumulates in steps. Intent is established. Details are checked. Accounts are validated. Each stage has its own requirements. Synthetic performance works best in short bursts. Staged systems ask for consistency over time.

Roles and separation of duties raise the bar quietly. Initiation, approval, and execution live with different people, often across different channels. An attacker now has to impersonate multiple individuals, in multiple ways, without contradiction. That kind of coordination is possible in theory and expensive in practice.

Identity verification moves from recognition to evidence. Hardware tokens. Out-of-band confirmation. Transaction-specific phrases that rotate. These methods don't rely on how something sounds or looks. They rely on possession, sequence, and proof that can't be improvised in real time.

Account validation shifts attention away from what's being said and toward what's been true over time. How long has this account existed? What kinds of flows normally pass through it? Does this change fit the counterparty's operating history? Verbal confirmation fades into the background. Corroboration carries the decision.

Execution choices complete the picture. Delay becomes a tool rather than an inconvenience. Rails are selected to match risk. When verification needs time, the system gives it time. When a transaction earns immediacy through preparation and proof, speed becomes safe.

Taken together, this is why automated fraud feels manageable inside structured systems. AI accelerates inconsistency. Informal processes strain under that pressure. Disciplined frameworks respond the same way every time, which makes them harder to exploit and easier to trust.

As attacks become more convincing, systems that rely on recognition fall away faster. Systems built on evidence become clearer, calmer, and more effective. Nothing here

requires predicting what the next fraud method will look like. It only requires committing to transactions that prove themselves before they execute.

Future-Proofing Area #2: New Transaction Rails and Technologies

Fraud isn't the only thing evolving. The way value moves is changing too.

Wires and ACH aren't going away, but they're no longer the whole story. We're already seeing API-driven money movement, real-time settlement, tokenized deposits, stablecoins, and on-chain transfers used alongside traditional rails. More will follow.

This is usually framed as a technology story. It's more accurate to see it as a continuity story. Rails change. The problem stays the same.

Every transaction—no matter how modern the tooling—still has to answer a few basic questions. Where does verification live? Who has authority? What evidence is required before value moves? And if something goes wrong, how does recovery work?

Those questions don't disappear when the rail is faster or more abstract. They become more important.

Classification still determines controls. Transaction staging still matters, even when settlement is instant. On-chain execution doesn't remove the need for intent, setup, and verification. It just compresses the execution window. The work shifts earlier. Evidence has to exist before the transaction is released into an environment where recovery may be limited or impossible.

Where organizations tend to struggle is not with any single new rail, but with how they respond to change. The instinct is to add tools or exception workflows. Over time, the transaction environment turns into a patchwork of systems that don't share context or rules.

The framework acts as the stabilizing layer.

When technology evolves underneath you, the organization doesn't have to. Teams don't relearn how to decide. Controls don't get reinvented. New rails plug into an existing order instead of disrupting it.

Future-Proofing Area #3: AI in Service of Structure

We started this chapter with AI as a threat: cloned voices, synthetic meetings, and $25 million gone in minutes. But that's only one side of what AI is doing to transactions.

The same technology that can make fraud faster can also be used for good. It can make preparation better. And in most cases, preparation is what's actually slowing down your transactions.

Before a single dollar moves, someone has to read the contracts, coordinate the participants, verify identities, confirm bank instructors, and reconcile the versions. That work is real and necessary, but most of it isn't financial judgment—it's coordination overhead.

An AI transaction coordinator addresses that problem directly. And no, I'm not talking about a chatbot or a digital assistant. This is an economic actor whose purpose is to prepare transactions for execution—reading, comparing, assembling, and verifying before anyone funds.

The result is something the industry hasn't had before: a compilable transaction.

For the software industry, code became transformative when it became compilable. A machine could verify that instructions were complete, consistent, and ready to run before execution.

Transactions can now approach that same standard. When every party is confirmed, every amount reconciled, every authorization in place, funding becomes a mechanical step rather than a judgment call made under pressure.

That shift benefits teams in three ways:

- First, it **reduces coordination costs**—the back-and-forth that slows a deal after the terms are already agreed.
- Second, it **elevates the people in the transaction**. When the system handles assembly, humans move from builders to approvers: setting goals, applying judgment, and authorizing outcomes instead of chasing documents.
- Third, **it changes what's insurable**. Fraud thrives in ambiguity—the gaps between what's been verified and what's been assumed. When a transaction is fully prepared, that ambiguity shrinks. Measurable risk is insurable risk.

AI coordination layers sit on top of well-structured transactions—they can't substitute for classification, staging, verification, and roles. The sequence still matters. But when that structure is finished, AI doesn't undermine it. It puts it to work.

The future of money movement is human-approved, AI-prepared transactions. Preparation handled by the system, judgment retained by the people.

Build for the Future of Transactions

When transactions are structured well, change loses its edge. New fraud techniques don’t arrive as existential threats. They show up as variations the system already knows how to handle. New rails don’t trigger panic or rushed adoption. They get evaluated calmly, against the same questions you’ve always asked.

The criminals who stole $25 million from Arup succeeded because they understood something the finance team didn't: in an era of synthetic media, strong frameworks become the only reliable defense.

Criminals pay attention to where verification is consistent, where authority is separated, where delays are intentional, and where evidence is required every time. Those environments are expensive to attack and hard to manipulate. Over time, they self-select away in favor of easier targets.

This is how durable systems win without drama.

Threats come and go. Tools rise and fall. Techniques have a shelf life. **What lasts is structure**.

Because when you can't trust your eyes and ears anymore, you better trust your process.

When you build transaction systems that stay strong longer than threats stay fashionable, you stop playing defense. You create infrastructure that lets people focus on building real things—closing deals, funding projects, growing organizations—without carrying constant background tension.

The future of transactions stops being something to fear. It becomes something you’re ready for.

CHAPTER 15: Implementation and Next Steps

Structure enables speed

At the beginning of this book, we talked about the Golden Gate Bridge as more than an engineering feat. It was a moment when disciplined work became something people could believe in.

By the time the bridge opened, the hardest parts were already invisible. The surveying. The sequencing. The decisions made early, when nothing yet existed to show for the effort. What people saw in 1937 was the span itself—clean lines, steady towers, a roadway that felt inevitable once it was there.

That's how well-built systems always feel in hindsight.

But that confidence didn't come from optimism. It came from structure. From acknowledging reality early and building in a way that could carry real weight. The bridge moved quickly because its foundation was right. Because risk was designed for instead of argued away.

Implementation works the same way. When people hear the word, they often picture paperwork, controls, or slowdown. What they miss is the moment on the other side of that work—when teams stop improvising and execution becomes calmer because the system is doing its job.

That feeling is earned. It comes from assembling pieces in the right order, and resisting the shortcuts that cause problems later. Once this structure is in place, you know it won't need to be reinvented every time conditions change.

The Golden Gate Bridge wasn't built to impress. It was built to last. The result just happened to be inspiring.

This final chapter is about doing the same with transactions. Taking what you've learned and turning it into something real. Something load-bearing. Something your organization can stand on with confidence—not just this year, but for decades to come.

Your First Action Step: Map Your Current State

Implementation doesn't start with tools. It doesn't start with vendors, platforms, or procedures.

It starts with honesty.

Most organizations already have a strong intuitive sense of where things feel fragile. People know which transactions create tension. They know where exceptions pile up. They know which approvals get rushed, which verifications get skipped when the clock is ticking.

Mapping what you actually do is the moment where that intuition becomes visible. Not to assign blame. Not to justify past decisions. Simply to understand the terrain you're standing on.

Map **transaction types**, not individual payments. You're not trying to document every invoice. You're trying to understand patterns of execution across closings, recurring vendor payments, multi-party transactions, and others.

For each type, write down what actually happens:

- Who initiates the transaction?
- Who approves it?
- What verification actually happens in practice?
- Where do exceptions show up?
- What changes under pressure?

When you do this honestly, two things happen. First, the gaps surface quickly—transaction types with no clear owner, approval authority that doesn't match reality, verification steps that depend on memory.

Second, once that reality is on paper, you have a clear starting line. The gaps are now fixable.

Joseph Strauss stood at that starting line. Before any steel was ordered for the Golden Gate Bridge, he and his team surveyed everything. Tides. Wind. Bedrock. Politics. Funding. They did this so they could build without surprises.

Mapping your current transactions is the same act. And once that terrain is clear, real progress can begin.

Build in the Order of Dependencies

Once reality is mapped, the temptation is to fix everything at once. The energy is understandable. But real progress comes from doing things in the right order, not doing more things simultaneously (that's how you burn out).

Transaction systems have dependencies. Some elements only work when others are already in place. When that order is ignored, controls get added and then reworked, exceptions multiply, and confidence erodes instead of building.

The framework is meant to be built sequentially.

Classification comes first—until there's shared agreement about what kind of transaction this is, every safeguard is operating in the dark. From there, **connections** become visible, **stages** give structure to how the transaction unfolds, and **authority** can be aligned to match how work actually moves. **Verification** only does valuable work once **roles** are clear. **Evidence** has to travel with the transaction or it degrades into memory. **Rail selection**—how money actually moves—is a security decision, and it only makes sense once everything before it is settled. **Insurance** and containment come last: they depend on the structure that precedes them.

In the Golden Gate Bridge construction, the order was never optional. The towers had to be anchored before the cables could be spun. The cables had to be tensioned before the roadway could hang. Each step carried the next. Skipping ahead wouldn't have saved time—it would have guaranteed rework.

The same logic applies here. When foundations come first, progress compounds. When they don't, effort turns into noise.

Stay Focused on the Destination

The destination shapes every implementation decision. So, what are you actually building toward?

When you fully implement the framework we've set out in this book, you create a transaction layer—a structured perimeter that sits between your organization and your bank. You don't need to reroute your banking relationship or introduce an intermediary that holds your funds. Your money stays in your accounts, your float stays yours, and your banking relationships stay intact.

What changes is the layer through which payment instructions pass before reaching the bank—verifying, structuring, and authorizing before anything executes.

Think of it the way a medieval city built walls. The market, institutions, and relationships all functioned the same. The walls didn't replace the city—they defined a perimeter. Everything that entered or left passed through a gate, and the gate had rules.

Most organizations today move money with no equivalent structure. Anyone who can manufacture a convincing instruction can set funds in motion without triggering a meaningful checkpoint. The transaction layer closes that space: it becomes the walls around your city.

This matters most for a specific category of payment: the irregular, high-stakes, instruction-based ones. Payroll runs through a processor. Recurring bills are on autopay. Those payments already have a system. What the transaction layer is built for is everything else—the one-off wires, the closing disbursements, the capital payments that arrive as instructions and leave by wire.

These transfers are too important to get wrong and too irregular to automate.

Inside the transaction layer is what we call the Execution Code. When a transaction is verified and ready to fund, the layer generates a unique code representing the entire flow of funds—every recipient, every amount, every authorization. The organization then sends one payment, to one place. The payment carries the code. The layer reads it, matches it to the verified record, and releases every payment in the flow simultaneously—each to a verified destination, covered by insurance, and recorded in the permanent audit trail. Counterparties don't need to change anything. Internal teams send money the same way they always have. What changes is what happens inside the layer before that wire goes out.

There's a dimension of this that rarely gets discussed but matters enormously to the people actually responsible for money movement. Right now, in most organizations, the liability for a fraudulent wire sits on a person—the CFO who approved it, the controller who sent it, the coordinator who verified the account over email. That's an invisible burden that makes closing days stressful.

When the system has verified the accounts, confirmed the amounts, documented the authorizations, and generated the Execution Code, human judgment at the moment of execution is no longer carrying the full weight of the transaction. The liability moves from your people to the policy. Insurance backed by institutional underwriters covers the transaction—not because someone bought a policy and hoped it would apply, but because the structure of the transaction itself is what makes it insurable. The verification happened. The record exists.

This is the shift the framework makes possible. Not from one bank to another. Not from one technology stack to another. From a system where security depends on individual vigilance—which is fragile, uninsurable, and unfair to the people carrying it—to one where it's built into the architecture of how money moves.

Follow Timelines that Respect Reality

Boards want reassurance. Executives want milestones. Teams want relief from known weaknesses. In that environment, it's easy to believe that a serious implementation should be finished quickly.

Durable systems don't work that way.

Large organizations can't transform everything in ninety days. Not because they lack effort or intent, but because structure takes time to align. Authority has to be clarified. Dependencies have to be agreed on. People have to understand not just what is changing, but why the order matters.

That doesn't mean nothing meaningful can happen early. Quite the opposite.

In the first phases, organizations can classify transactions clearly, map real workflows, identify the highest-risk gaps, align leadership on priorities, and build shared agreement about what "good" looks like. That work doesn't always produce visible controls right away, but it creates momentum that lasts. It replaces anxiety with direction.

The Golden Gate Bridge finished ahead of schedule. Not because the plan was aggressive, but because it was honest. The timeline accounted for weather, tides, engineering limits, and human risk. No one promised a miracle and then tried to outrun reality.

Implementation benefits from the same discipline. When timelines respect reality, progress becomes cumulative instead of cyclical. Each phase builds confidence for the next.

The goal isn't speed for its own sake. A system built at a pace people can maintain will outperform a rushed one—not just in outcomes, but in trust.

Develop the Secret Multiplier: Culture

Even the best systems only work if people trust them.

Not trust in the abstract sense, but the practical kind. Trust that the system will back them up when they slow something down. Trust that raising a hand won't turn them into the problem. Trust that verification is there to protect the work, not question their judgment.

This is where culture becomes a multiplier.

In many organizations, speed is what gets celebrated. Completions are visible. Pauses are not. Over time, that sends a message: moving fast is valued, stopping is suspect. Even good controls start to feel like obstacles when the social incentives point in the other direction.

Strong systems change that signal. They make stopping part of the job. Verification becomes professionalism, not friction. These systems reward the person who speaks up early, especially when it's uncomfortable.

In the organizations I've worked with that do this well, that tone is very clearly set from the top. When leaders signal that a short pause is preferable to a long cleanup, behavior follows. Teams relax, and communication improves.

When building the Golden Gate Bridge, the workers faced the inherent dangers of balancing on narrow beams, hundreds of feet above the bay, fighting wind and fog. A single misstep could be fatal. Early in the project, men were dying.

Joseph Strauss made a decision that many thought was excessive. He ordered a massive safety net stretched beneath the span, the first of its kind used on a bridge of that scale. It was expensive. It took time to install. Critics worried it would slow the work.

It didn't.

The net caught nineteen men who fell during construction. They called themselves the Halfway to Hell Club. What mattered even more than the lives saved was what changed on the job site afterward.

The presence of the net didn't make the work easier. It made it sustainable. Workers knew that if something went wrong, the system was there to catch them. That confidence changed behavior. People worked more deliberately. Risks were taken thoughtfully, not bravely. The project moved faster because fewer mistakes turned into tragedies.

Transaction systems work the same way.

When people know the framework will back them up—when verification is expected, when stops are respected, when evidence matters more than tone—they stop feeling like they have to choose between doing the right thing and getting the work done. The system becomes a net: something that makes difficult work sustainable over time.

Test Before Crisis

Confidence is built on the foundation of proof.

When the Golden Gate Bridge opened in May of 1937, the first day wasn't reserved for cars. Nearly two hundred thousand people walked the span on foot. Families. Workers. Children. They leaned over the railings. They stopped in the middle. They trusted the structure enough to linger.

That confidence didn't come from hope. It came from preparation.

By the time anyone set foot on the roadway, the bridge had already been tested repeatedly. Loads had been applied. Cables tensioned and re-tensioned. Components stressed under conditions more severe than daily use would ever demand. The public wasn't being asked to believe. They were stepping onto something that had already proven it could hold.

That's the role testing plays in transaction systems.

Teams don't trust systems because they've read the documentation. They trust them because they've seen the system hold when it's supposed to. Testing is how that trust gets built—quietly, deliberately, and before anything is at stake.

Too many organizations only learn how their controls behave during a real incident. By then, the pressure is high, the consequences are real, and every gap feels personal. Testing earlier changes that dynamic. It turns unknowns into information and replaces anxiety with familiarity.

Effective testing doesn't require elaborate simulations. It focuses on a few simple questions: *Will the system say no when it should? Will it slow down when something doesn't fit? Will it hold its shape under stress?*

That means testing scenarios such as:

- Transactions that should be rejected because they fall outside classification or staging rules
- Impersonation attempts that try to bypass identity verification
- Suspicious or inconsistent account details that don't match historical behavior
- Approval attempts made by people who shouldn't have the authority to act

Each test reinforces the same message: the system is paying attention.

These exercises also surface practical issues that policy reviews miss. You may discover that emergency workflows bypass verification. That approvals work during business hours but fail on weekends. That certain roles can override controls without realizing it. Finding those gaps early isn't a setback. It's part of the work.

The important shift is when teams stop wondering how the system will behave and start knowing. That's when execution changes. People rely on structure instead of instinct. And when something unusual appears, the response is steadier because the system has already shown what it does under strain.

Build the Bridge That Lasts

No transaction system will stop every attack. No structure eliminates risk entirely. What durable systems do is narrow the field—catch most problems early, contain damage when something slips through, and let legitimate work move quickly without asking people to gamble.

When starting the Golden Gate project, Joseph Strauss could have chosen a different path. A cheaper bridge. A faster build. A structure that met the minimum requirements and hoped for calm weather.

Instead, he chose to build the right bridge. One designed to endure wind, fog, tides, and time. The result still carries millions of people across the bay nearly a century later.

That choice is available here too.

You already understand the risks. You've seen where transactions break. You now have the architecture to strengthen those weak points.

Implementation is the act of choosing to build well.

Not the minimum that satisfies an audit. Not the fastest path that gets through this quarter. But the structure that keeps working when conditions change.

Transactions don't have to be fragile. Speed doesn't have to be dangerous. Discipline doesn't have to slow progress. When the structure is right, work becomes calmer. Decisions get easier. Trust compounds. The system stops needing constant attention because it's doing what it was designed to do.

So, build your bridge. Make it strong enough to last longer than the threat of the moment.

When the fog lifts, it will still be standing.

z